

Vorabfragen

- Ein klassisches neuronales Netz ist vollständig verbunden. Welche Probleme könnten auftreten, wenn man damit direkt Bilder (z. B. 256×256 Pixel) verarbeitet?
- Im menschlichen visuellen System reagieren Neuronen nur auf kleine Bildbereiche. Warum könnte dieses Prinzip für die automatische Bilderkennung nützlich sein?
- Stellen Sie sich vor, Sie möchten in Bildern Kanten erkennen. Wie könnte ein Algorithmus erkennen, ob in einem Bild eher horizontale oder vertikale Strukturen vorkommen?



Deep Learning, Machine Learning und Künstliche Intelligenz – SS 26

Gelesen von Prof. Dr. Daniel Gaida

02.06.2026

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Technology
Arts Sciences
TH Köln

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können ein Deep Learning Projekt zur Bildklassifikation selbstständig durchführen, indem sie

- für eine gegebene Anwendung ein geeignetes convolutional neuronales Netz in keras implementieren,
- trainieren, validieren und testen
- und bei schlechten Validierungsergebnissen Lösungsstrategien zur Verbesserung der Ergebnisse anwenden können,

um später eigene Deep Learning Projekte zur Bilderkennung umsetzen zu können.

Überprüfung des Lernziels: S. fortlaufende Übung heute.

Lernraum II: Deep Learning

- Künstliche neuronale Netze
 - Grundlagen
 - Training
 - Implementierung
 - Lösungsstrategien, um gute Ergebnisse zu erzielen
- -
- -
- -
- -
- -
- -
- -

Lernraum II: Deep Learning

- Künstliche neuronale Netze
 - Grundlagen
 - Training
 - Implementierung
 - Lösungsstrategien, um gute Ergebnisse zu erzielen
- **Convolutional neuronale Netze**
 - **Bildererkennung**
 -
 -
 -
 -
 -
 -
 -

Lernraum II: Deep Learning

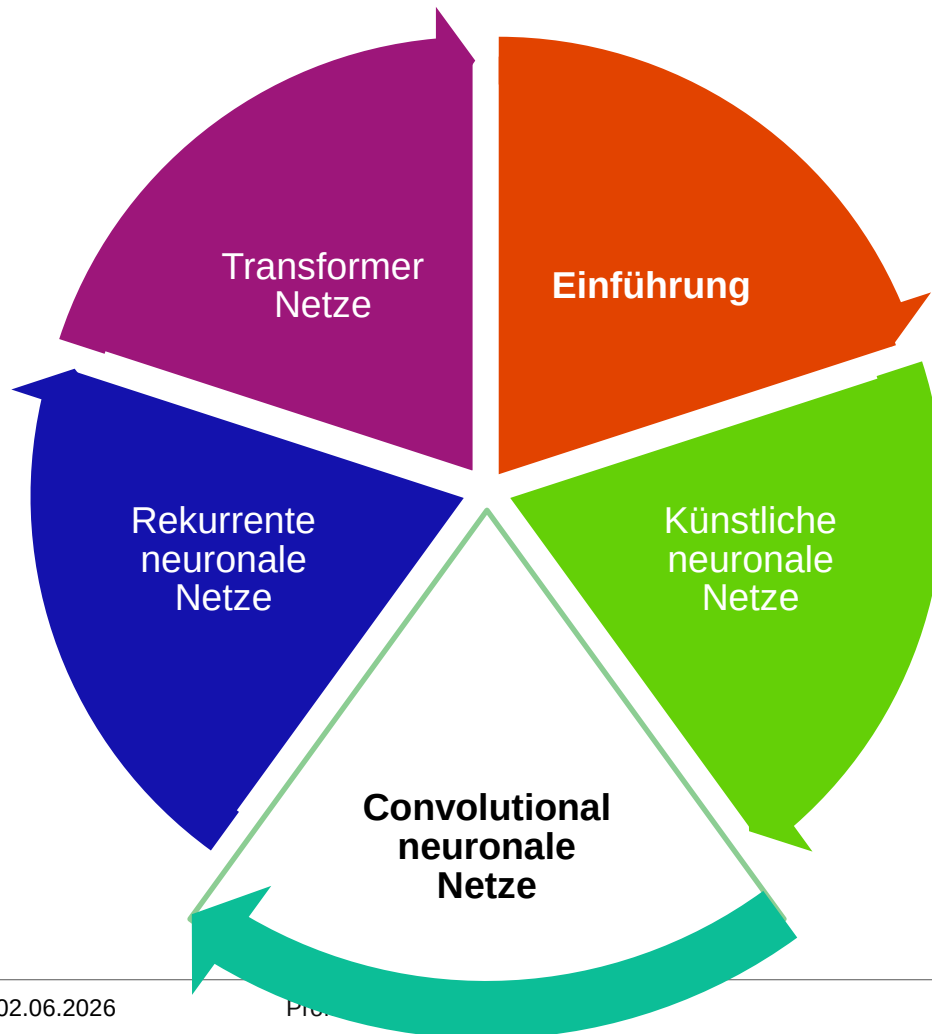
- Künstliche neuronale Netze
 - Grundlagen
 - Training
 - Implementierung
 - Lösungsstrategien, um gute Ergebnisse zu erzielen
- **Convolutional neuronale Netze**
 - **Bildererkennung**
- Rekurrente neuronale Netze
 - Zeitreihen, Text
- Transformer Netze
 - Text, Bildererkennung
- -
 -

Lernraum II: Deep Learning

- Künstliche neuronale Netze
 - Grundlagen
 - Training
 - Implementierung
 - Lösungsstrategien, um gute Ergebnisse zu erzielen
- **Convolutional neuronale Netze**
 - **Bilderkennung**
- Rekurrente neuronale Netze
 - Zeitreihen, Text
- Transformer Netze
 - Text, Bilderkennung
- Anwendungen
 - Entwickeln eines Chatbots
 - Objektdetektion und -segmentierung

Deep Learning

Ausblick



Convolutional neuronale Netze (CNN)

- Der visuelle Cortex
- Aufbau eines CNNs
 - Rezeptive Felder
 - Filter
 - Feature Maps
 - Poolingschicht
- Beispielnetz: LeNet-5
- Keras Code

Convolutional Neural Network - CNN

Einführung



CNNs in der optischen Verarbeitung

- Unterscheidung von Objekten
- Autonomes Fahren
- Augmented Reality
- Gesichtserkennung

-
-

Convolutional Neural Network - CNN

Einführung



CNNs in der optischen Verarbeitung

- Unterscheidung von Objekten
- Autonomes Fahren
- Augmented Reality
- Gesichtserkennung

Außerdem

- Stimmenerkennung
- Audioverarbeitung über Analyse des Spektrogramms

Convolutional Neural Network - CNN

Leitfragen:

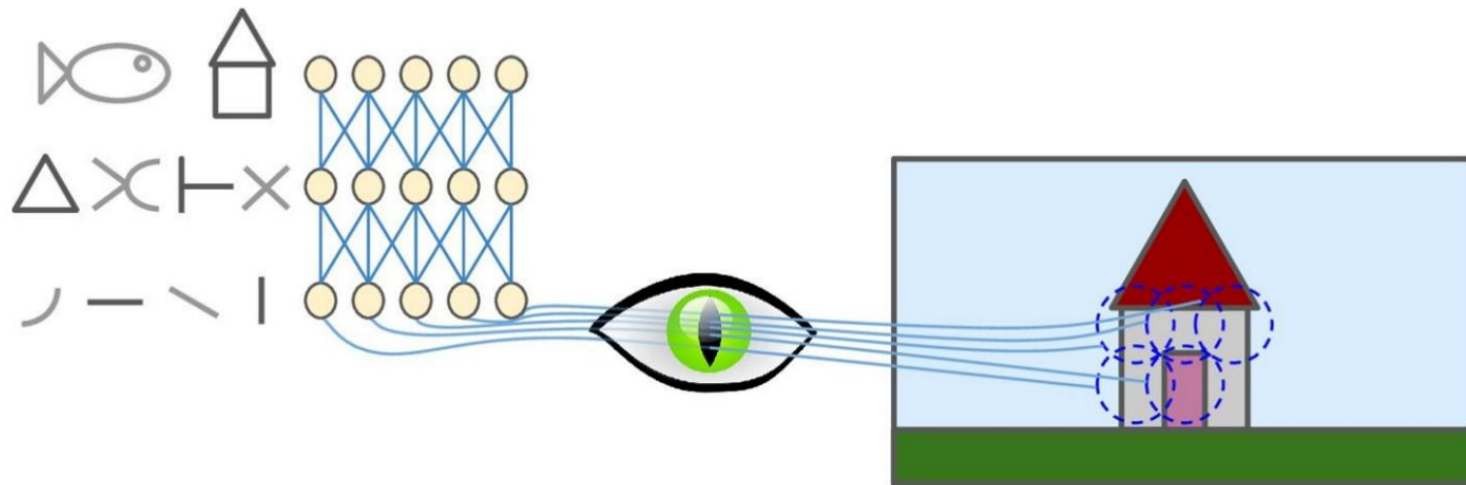
- Wie funktioniert der visuelle Kortex?
- Wie wird das biologische Vorbild in einem Convolutional neural network umgesetzt?

Convolutional Neural Network - CNN

Rezeptive Felder im visuellen Cortex

Visueller Cortex

- Neuronen verfügen über ein kleines rezeptives Feld
- Visuelle Reize werden nur in dieser begrenzten Region verarbeitet
- Können überlappen und decken zusammen das vollständige Bild ab
-
-

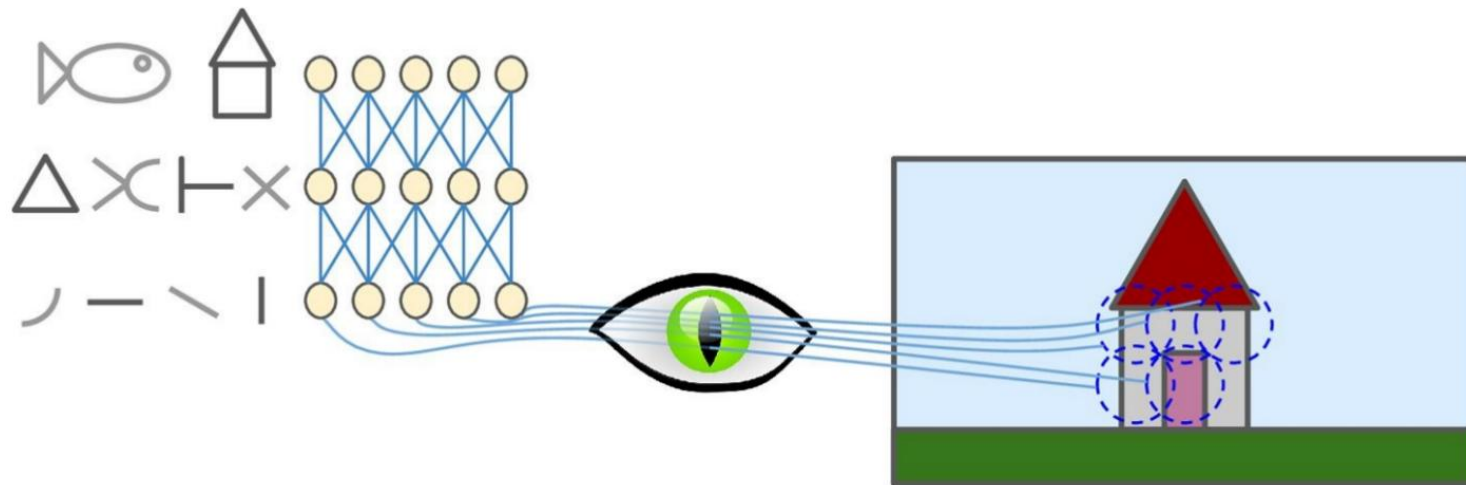


Convolutional Neural Network - CNN

Rezeptive Felder im visuellen Cortex

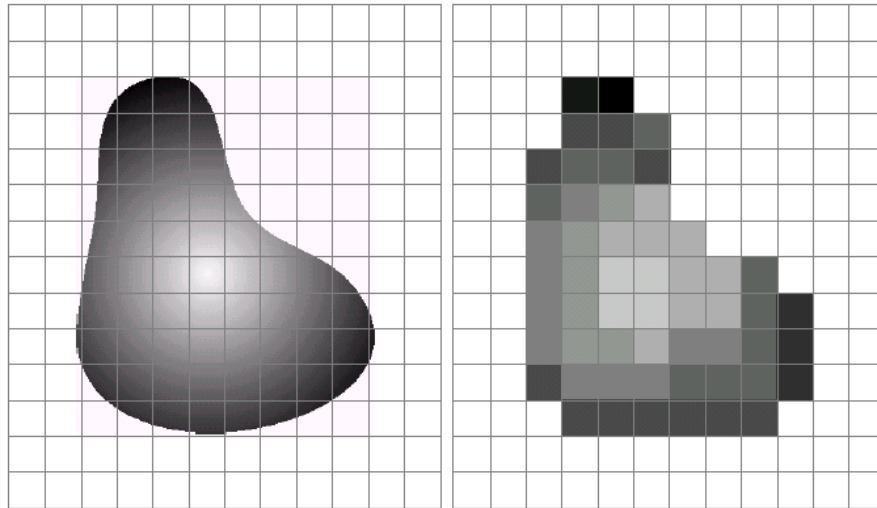
Visueller Cortex

- Neuronen verfügen über ein kleines rezeptives Feld
- Visuelle Reize werden nur in dieser begrenzten Region verarbeitet
- Können überlappen und decken zusammen das vollständige Bild ab
- Verschiedene Neuronen reagieren auf verschiedene Muster / Linien
- Higher-level Neuronen reagieren auf die Ausgänge der lower-level Neuronen



Wie wird ein Grauwertbild im Computer abgebildet?

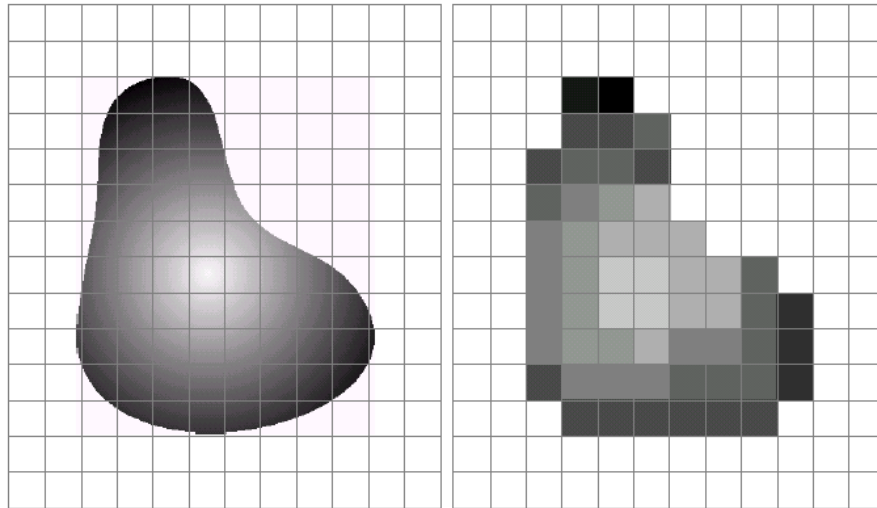
Eine Matrix von Intensitätswerten



Wie wird ein Grauwertbild im Computer abgebildet?

Eine Matrix von Intensitätswerten

- Üblicherweise wird ein Byte pro Intensitätswert verwendet:
 - 0 = schwarz, 255 = weiß



=

255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	20	0	255	255	255	255	255	255	255
255	255	255	75	75	75	255	255	255	255	255	255
255	255	75	95	95	75	255	255	255	255	255	255
255	255	96	127	145	175	255	255	255	255	255	255
255	255	127	145	175	175	175	255	255	255	255	255
255	255	127	145	200	200	175	175	95	255	255	255
255	255	127	145	200	200	175	175	95	47	255	255
255	255	127	145	145	175	127	127	95	47	255	255
255	255	74	127	127	127	95	95	95	47	255	255
255	255	255	74	74	74	74	74	74	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255

Convolutional Neural Network - CNN

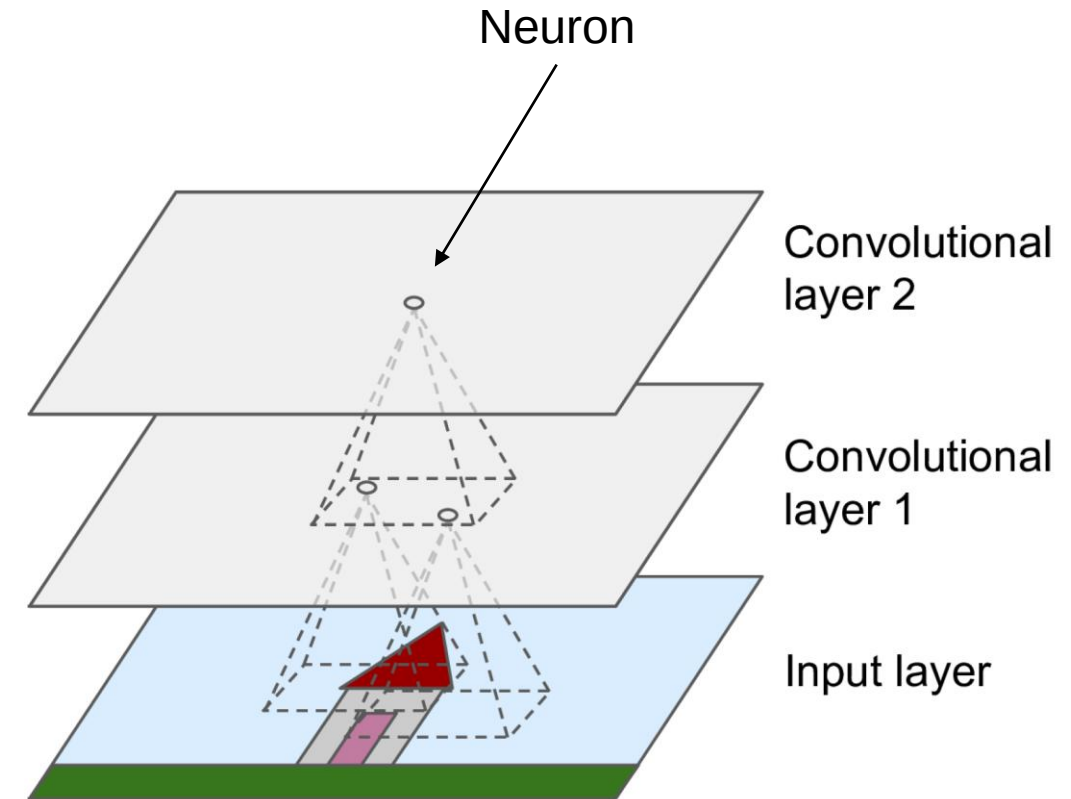
Aufbau eines CNN

Die Eingabeschicht

- Neuronen der Eingabeschicht
 - Verbunden mit Bild
 - Es gibt so viele Neuronen wie es Pixel gibt
 - Neuronen liefern nur das Eingangssignal weiter

■

■

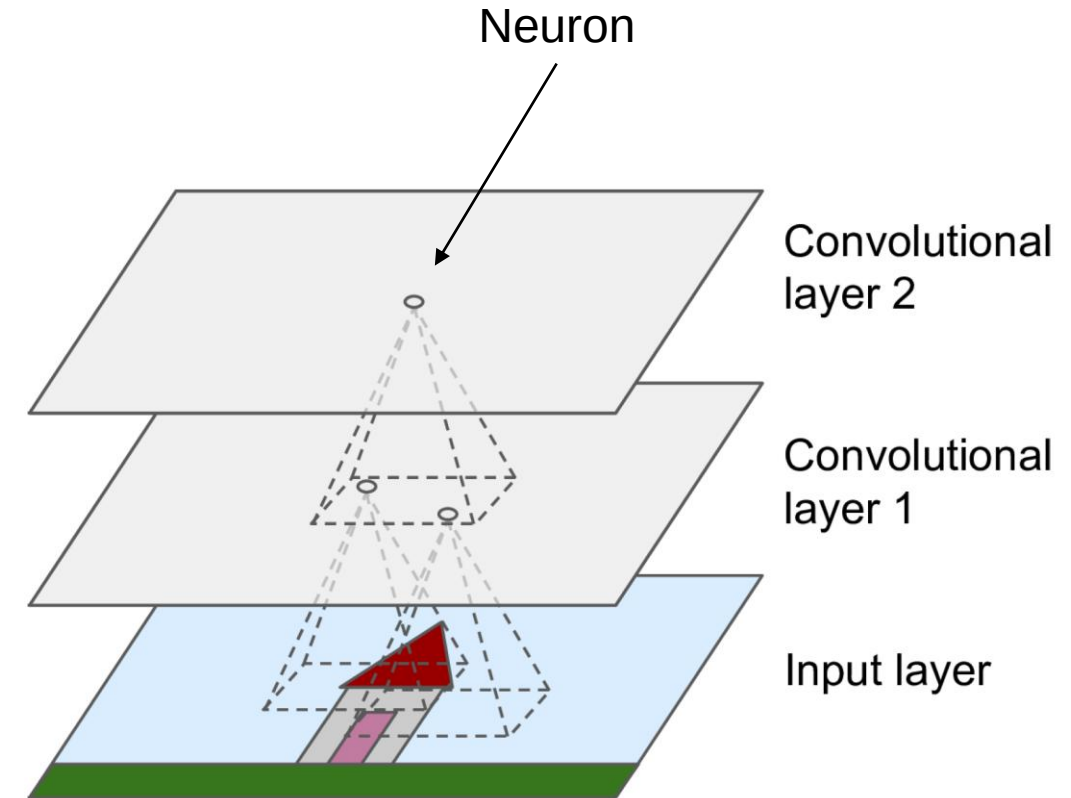


Convolutional Neural Network - CNN

Aufbau eines CNN

Die Eingabeschicht

- Neuronen der Eingabeschicht
 - Verbunden mit Bild
 - Es gibt so viele Neuronen wie es Pixel gibt
 - Neuronen liefern nur das Eingangssignal weiter
- Rezeptives Feld eines Neurons in Schicht 1 wird durch Quadrat in Eingangsschicht visualisiert
-

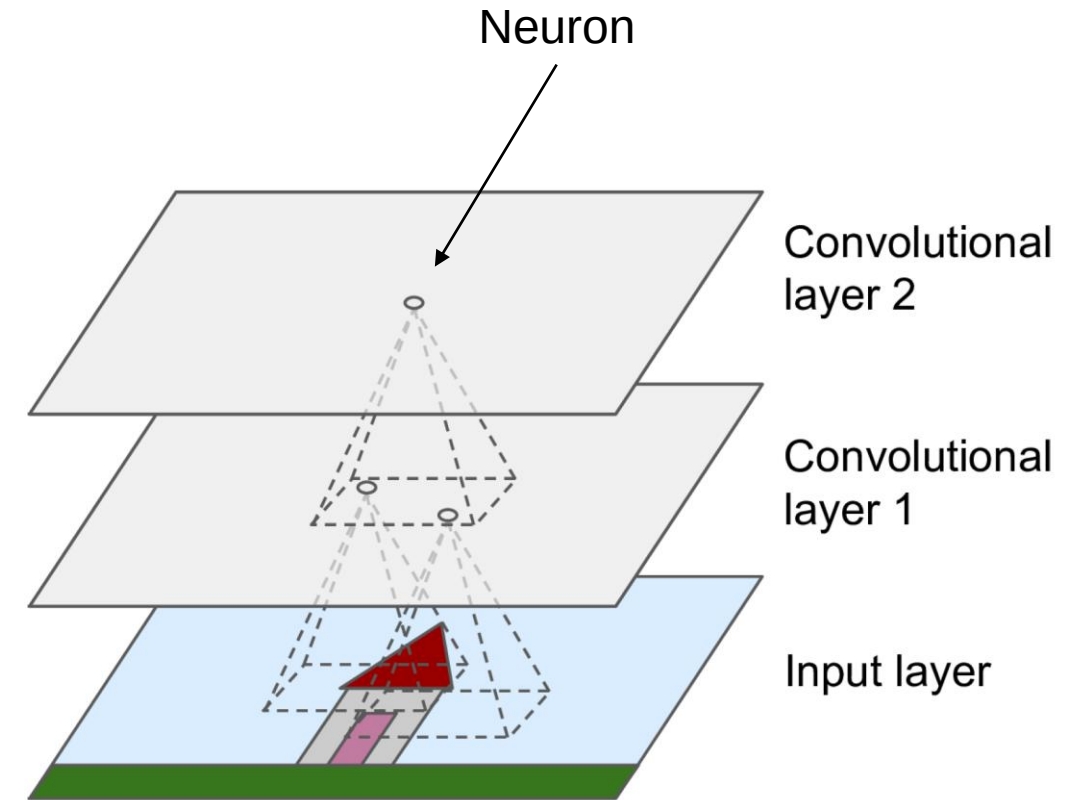


Convolutional Neural Network - CNN

Aufbau eines CNN

Die Eingabeschicht

- Neuronen der Eingabeschicht
 - Verbunden mit Bild
 - Es gibt so viele Neuronen wie es Pixel gibt
 - Neuronen liefern nur das Eingangssignal weiter
- Rezeptives Feld eines Neurons in Schicht 1 wird durch Quadrat in Eingangsschicht visualisiert
- Neu: bisher war ein Neuron immer mit allen Neuronen der vorherigen Schicht verbunden (vollvernetzt). Jetzt nur verbunden mit Neuronen im rezeptiven Feld



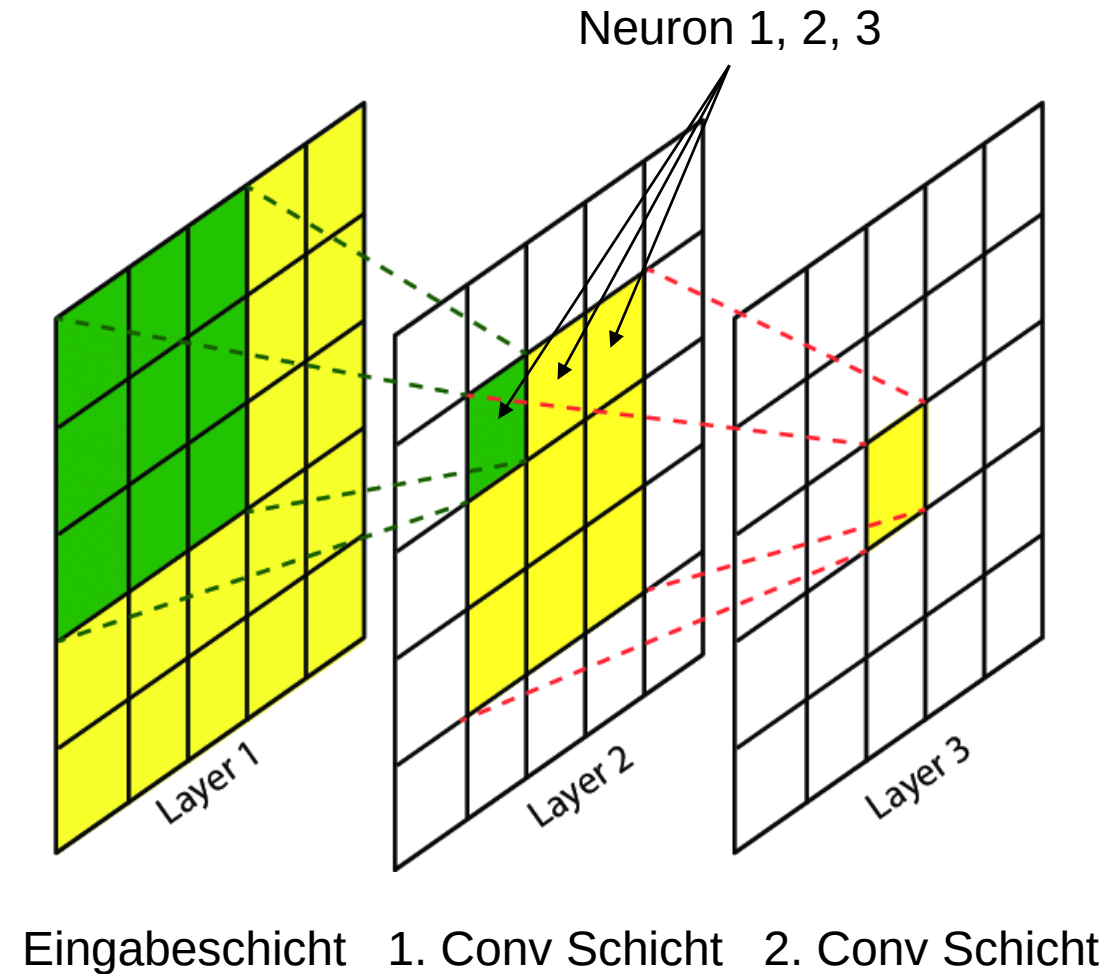
Convolutional Neural Network - CNN

Aufbau eines CNN

Die Convolutional Schicht

- Neuronen der ersten Convolutional Schicht
 - Verbunden mit Eingangsneuronen aus ihrem rezeptiven Feld (Viereck)
 - Erkennung von **Low-level Features** (Linien, Kurven,...)

-
-
-
-

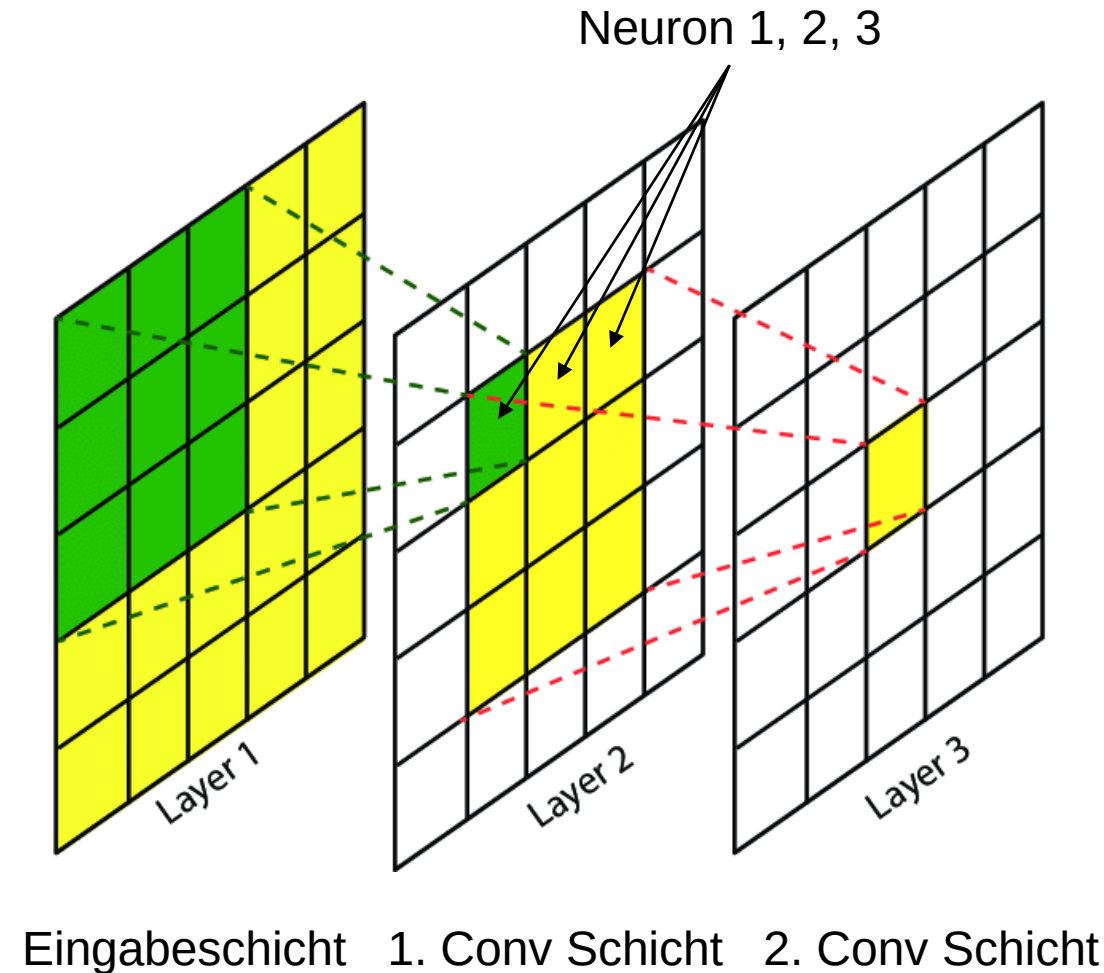


Convolutional Neural Network - CNN

Aufbau eines CNN

Die Convolutional Schicht

- Neuronen der ersten Convolutional Schicht
 - Verbunden mit Eingangsneuronen aus ihrem rezeptiven Feld (Viereck)
 - Erkennung von **Low-level Features** (Linien, Kurven,...)
- Wie bisher, berechnet jedes Neuron eine gewichtete Summe seiner Eingänge
-
-
-

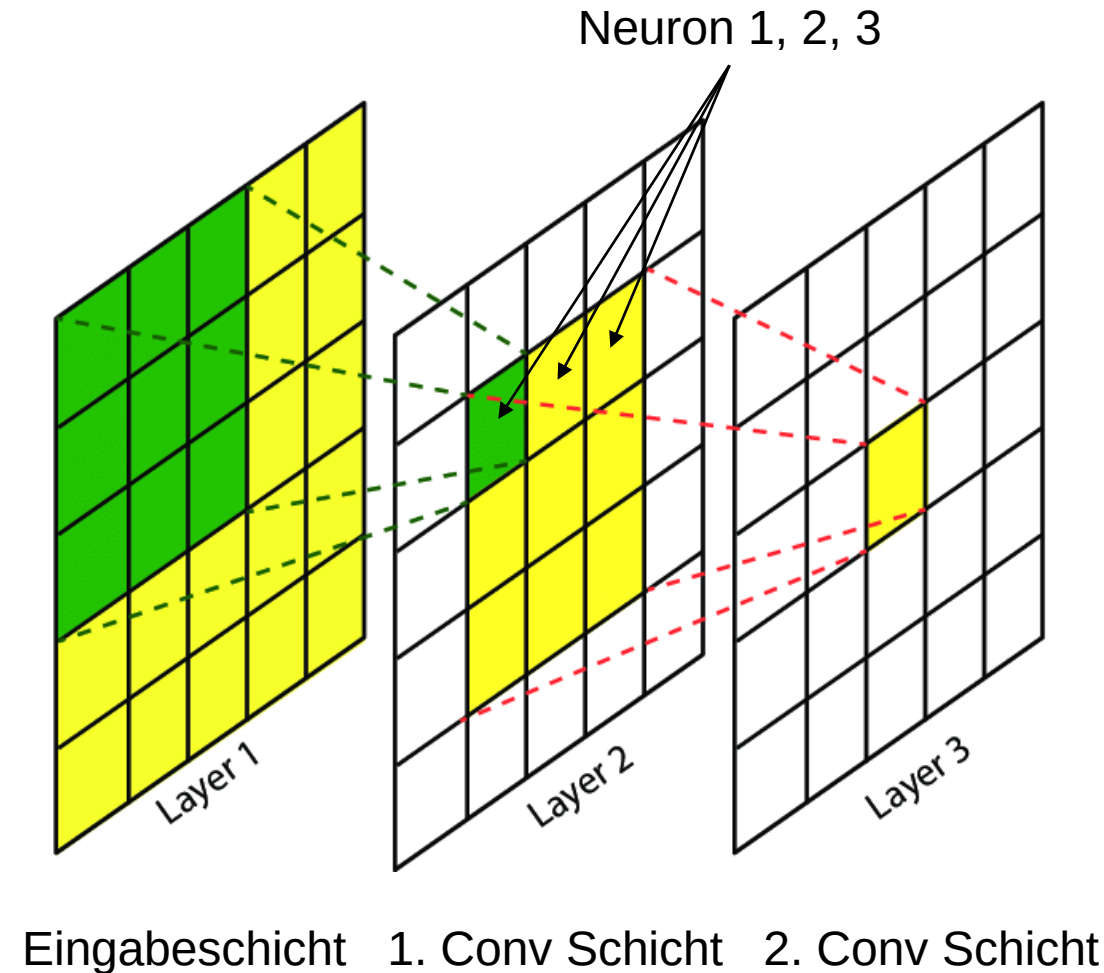


Convolutional Neural Network - CNN

Aufbau eines CNN

Die Convolutional Schicht

- Neuronen der ersten Convolutional Schicht
 - Verbunden mit Eingangsneuronen aus ihrem rezeptiven Feld (Viereck)
 - Erkennung von **Low-level Features** (Linien, Kurven,...)
- Wie bisher, berechnet jedes Neuron eine gewichtete Summe seiner Eingänge
- Neu: Alle Neuronen in einer Schicht teilen sich die Gewichte und den Bias!
-
-

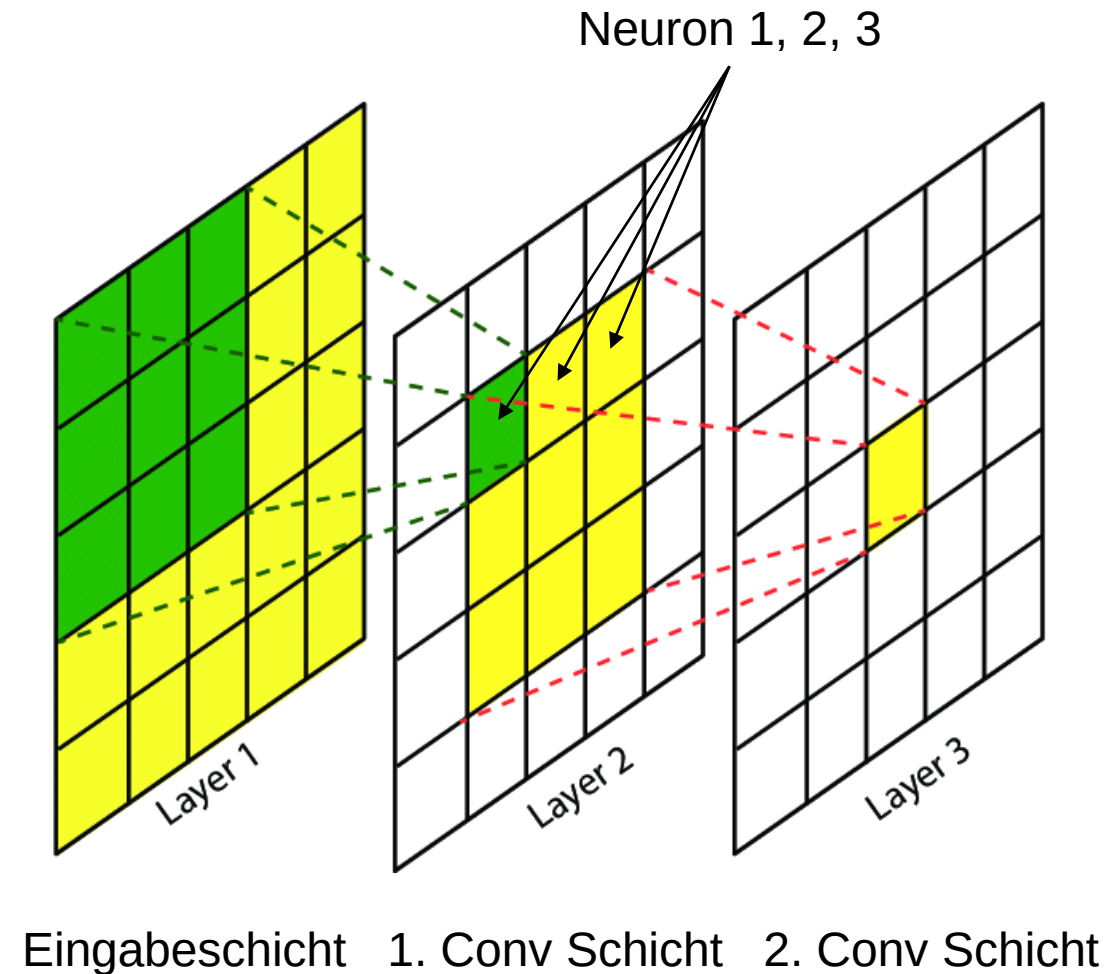


Convolutional Neural Network - CNN

Aufbau eines CNN

Die Convolutional Schicht

- Neuronen der ersten Convolutional Schicht
 - Verbunden mit Eingangsneuronen aus ihrem rezeptiven Feld (Viereck)
 - Erkennung von **Low-level Features** (Linien, Kurven,...)
- Wie bisher, berechnet jedes Neuron eine gewichtete Summe seiner Eingänge
- Neu: Alle Neuronen in einer Schicht teilen sich die Gewichte und den Bias!
- Gewichte definieren welche Muster ein Neuron erkennen kann (warum, s. später)

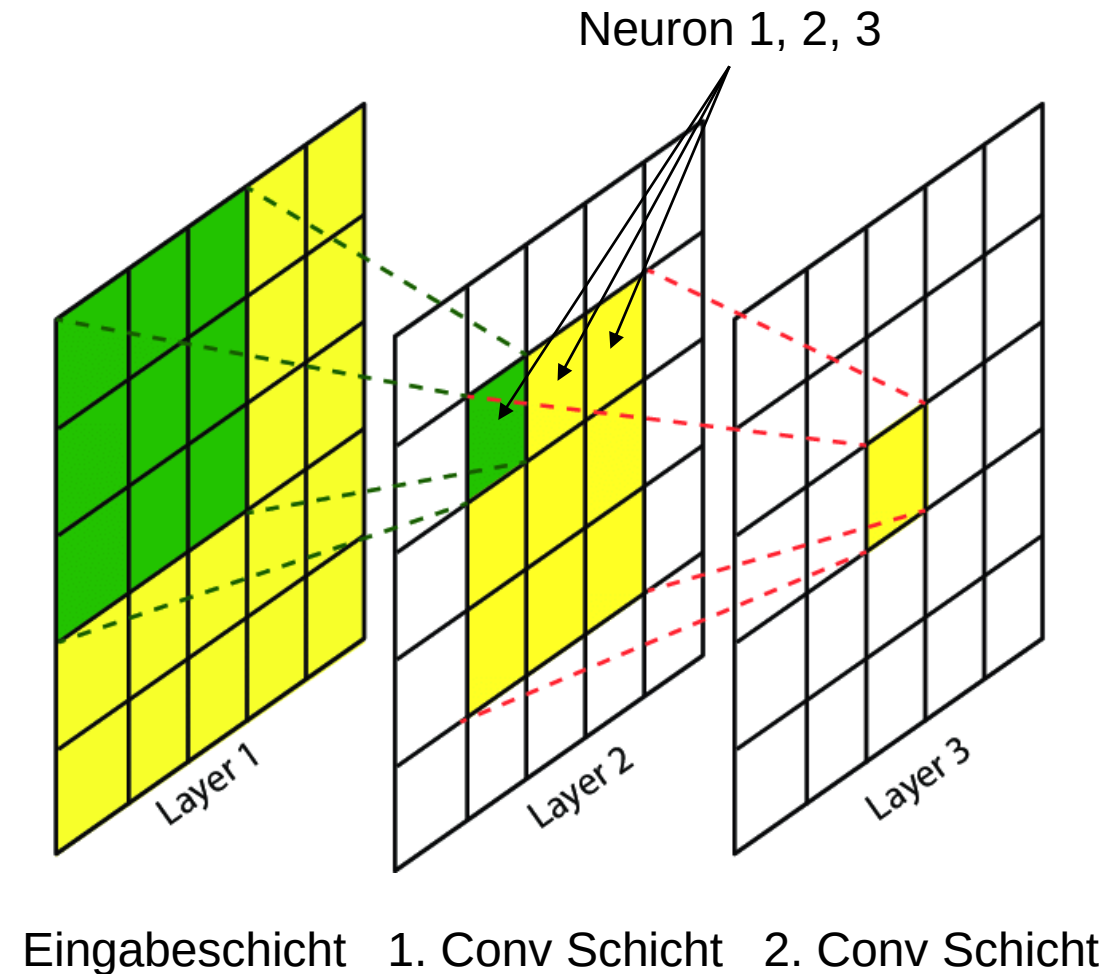


Convolutional Neural Network - CNN

Aufbau eines CNN

Die Convolutional Schicht

- Neuronen der ersten Convolutional Schicht
 - Verbunden mit Eingangsneuronen aus ihrem rezeptiven Feld (Viereck)
 - Erkennung von **Low-level Features** (Linien, Kurven,...)
- Wie bisher, berechnet jedes Neuron eine gewichtete Summe seiner Eingänge
- Neu: Alle Neuronen in einer Schicht teilen sich die Gewichte und den Bias!
- Gewichte definieren welche Muster ein Neuron erkennen kann (warum, s. später)
- → Eine Convolutional Schicht erkennt nur eine Art von Muster (bspw. horizontale Kanten)

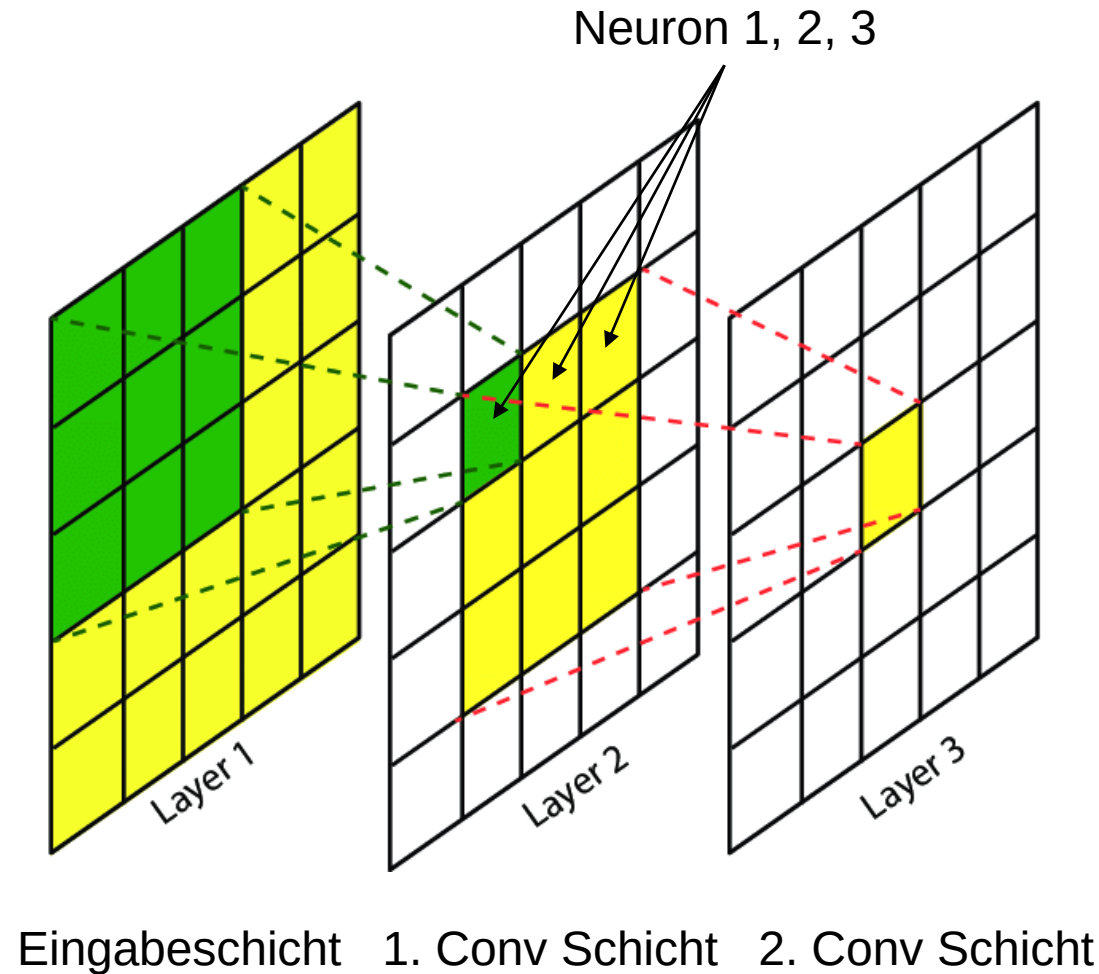


Convolutional Neural Network - CNN

Aufbau eines CNN

Die Convolutional Schicht

- Ausgabewerte der Neuronen einer Convolutional Schicht ergeben ein Bild
- In der Abbildung rechts werden die Schichten als diese Bilder dargestellt

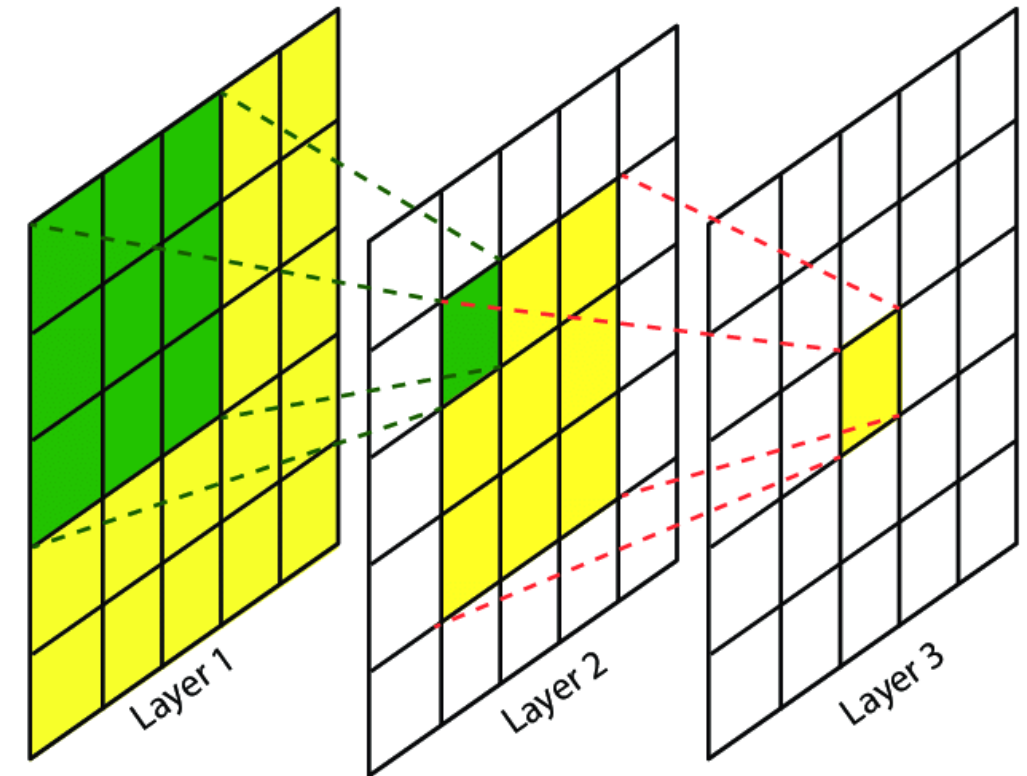


Convolutional Neural Network - CNN

Aufbau eines CNN

Die Convolutional Schicht

- Neuronen aus der zweiten Conv. Schicht
 - Verbunden mit Neuronen aus der ersten Conv. Schicht aus ihrem rezeptiven Feld
 - Erkennung von Features zusammengesetzt aus der vorherigen Schicht.
 - **High-level Features** (Augen, Ohren, Mund, Pfoten,...)
-
-



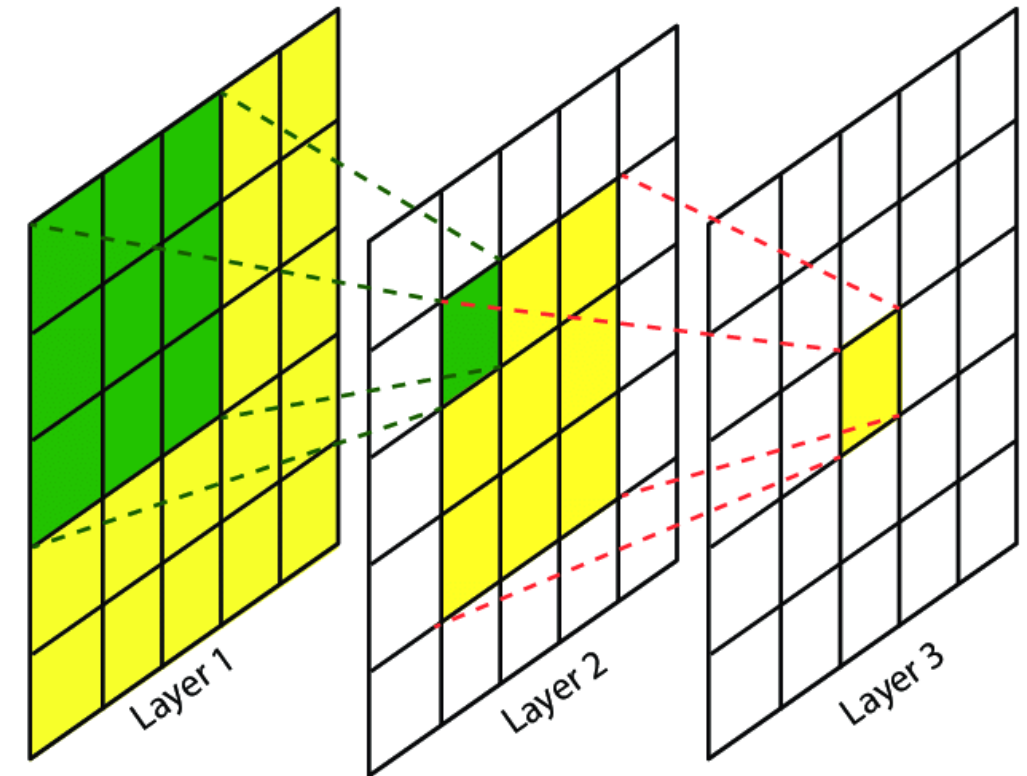
Eingabeschicht 1. Conv Schicht 2. Conv Schicht

Convolutional Neural Network - CNN

Aufbau eines CNN

Die Convolutional Schicht

- Neuronen aus der zweiten Conv. Schicht
 - Verbunden mit Neuronen aus der ersten Conv. Schicht aus ihrem rezeptiven Feld
 - Erkennung von Features zusammengesetzt aus der vorherigen Schicht.
 - **High-level Features** (Augen, Ohren, Mund, Pfoten,...)
- Schema kann beliebig oft wiederholt werden
- Erkennung von sehr komplexen Mustern



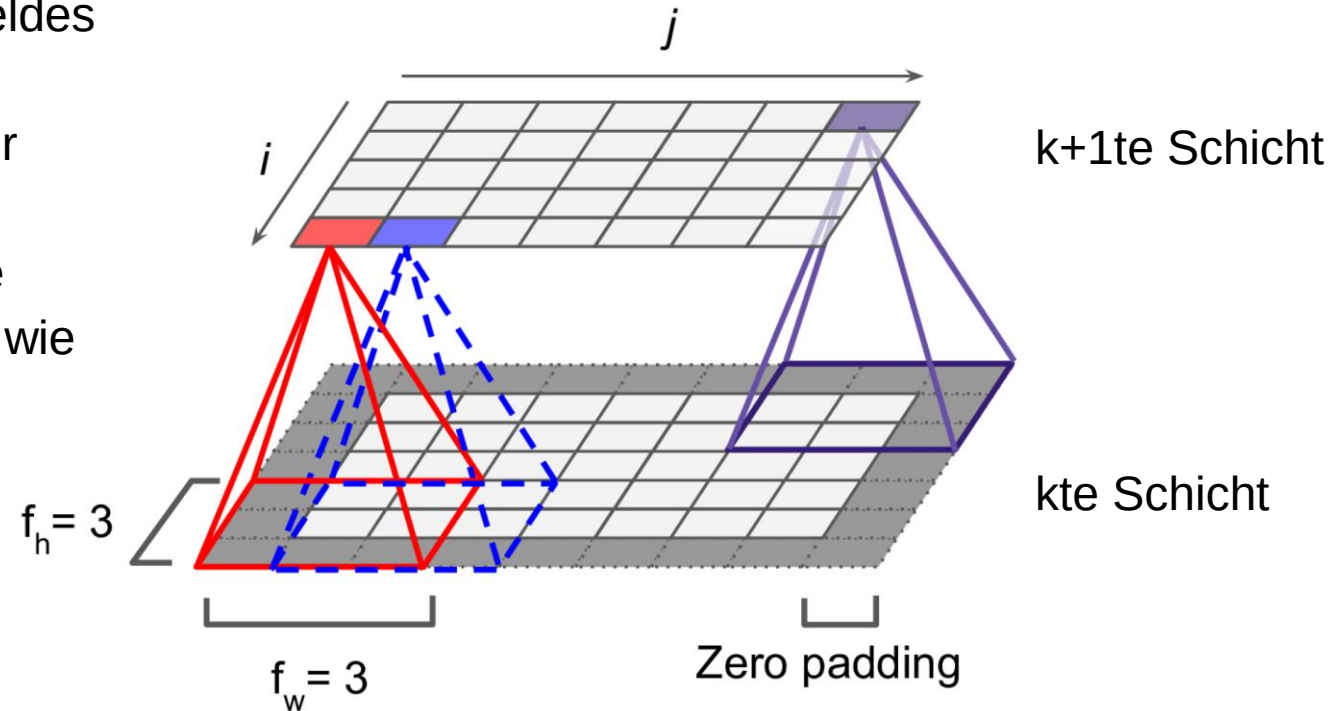
Eingabeschicht 1. Conv Schicht 2. Conv Schicht

Convolutional Neural Network - CNN

Aufbau eines CNN

Parameter der Convolutional Schicht

- Breite (f_w) und Höhe (f_h) des rezeptiven Feldes ist 3
- Stride: Verschiebung der rezeptiven Felder gegeneinander
- Zero Padding: einfügen von Nullen um die Eingänge, damit Ausgangsbild so groß ist wie Eingangsbild

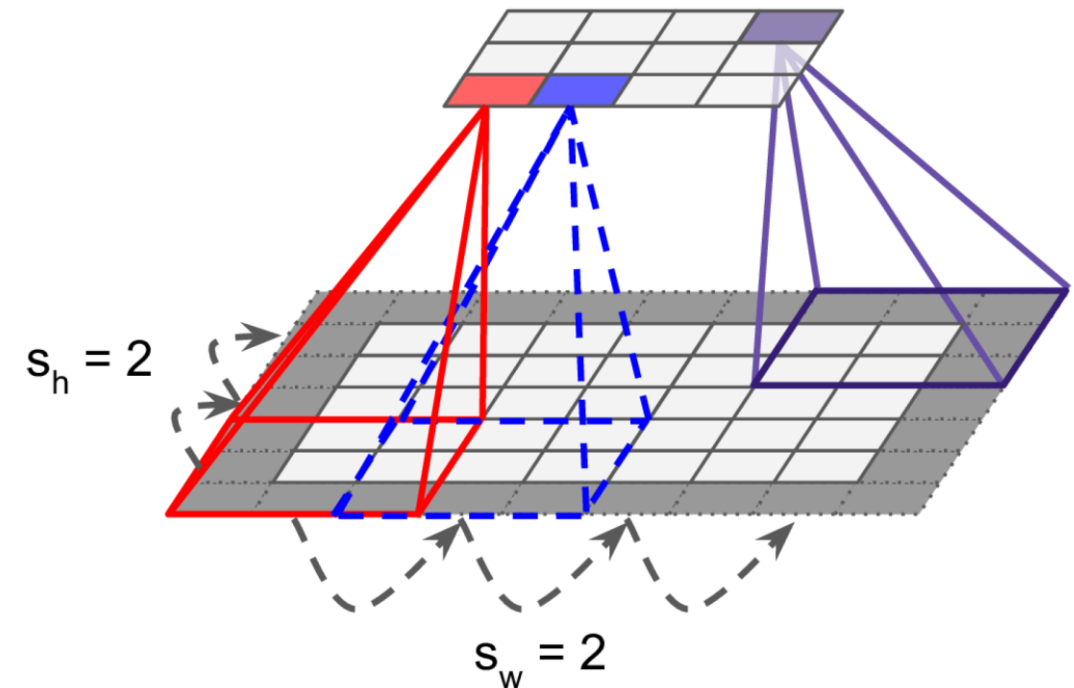


Convolutional Neural Network - CNN

Aufbau eines CNN

Parameter der Convolutional Schicht

- Breite (f_w) und Höhe (f_h) des rezeptiven Feldes ist 3
- Stride: Verschiebung der rezeptiven Felder gegeneinander (hier: $s_h = s_w = 2$)
 - Verkleinert das Ausgangsbild



Convolutional Neural Network - CNN

Aufbau eines CNN

Leitfragen:

- Wie erkennt ein Neuron horizontale/vertikale Kanten?

Convolutional Neural Network - CNN

Aufbau eines CNN

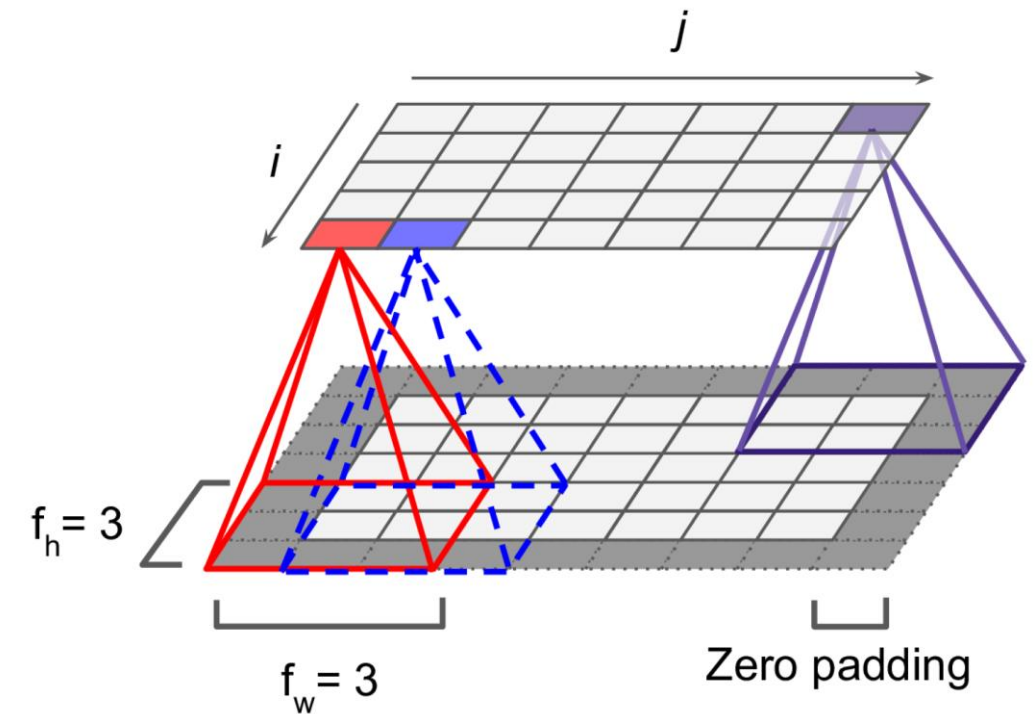
Filter (bzw. Kernel):

- Vektor von Gewichten, visualisiert als Matrix in der Dimension des rezeptiven Felds
- Speichert das Muster bzw. Template
-
-

Gewicht 1 →

Gewicht 4 →

1	0	-1
1	0	-1
1	0	-1



Convolutional Neural Network - CNN

Aufbau eines CNN

Filter (bzw. Kernel):

- Vektor von Gewichten, visualisiert als Matrix in der Dimension des rezeptiven Felds
- Speichert das Muster bzw. Template
- Pixel des rezeptiven Felds werden mit dem Filter multipliziert (gewichtete Summe)
 - Bestimmung der Ähnlichkeit zwischen Eingabe und gespeichertem Template

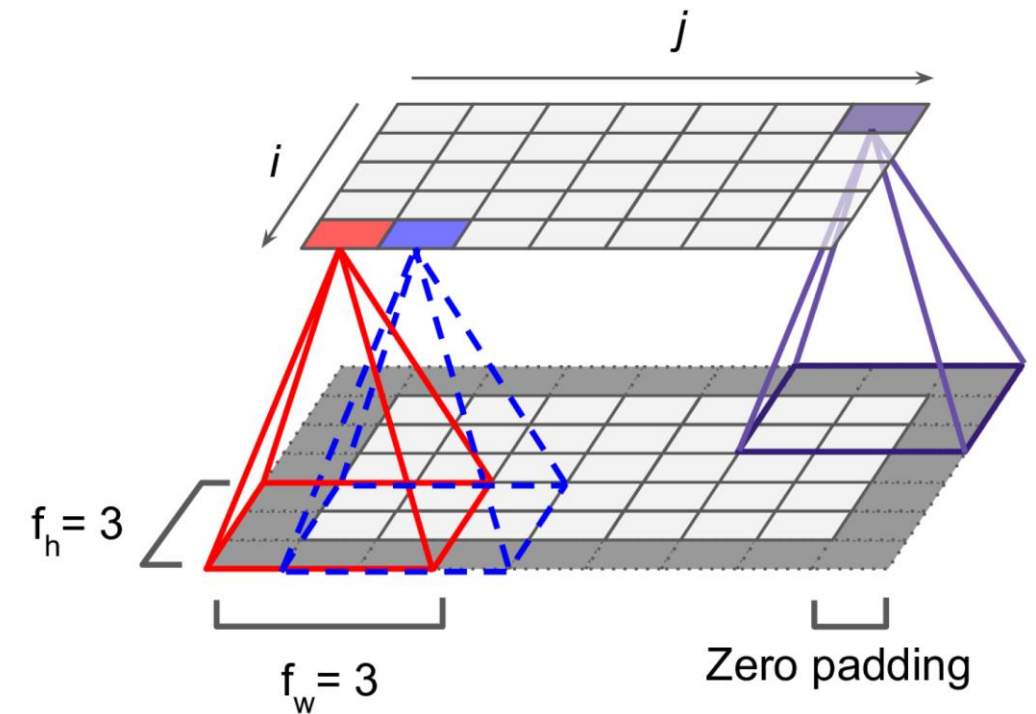
Gewicht 1 →

1	0	-1
1	0	-1
1	0	-1

Gewicht 4 →

Rezeptives Feld:

250	250	1
233	255	0
227	254	2



Convolutional Neural Network - CNN

Aufbau eines CNN

Filter (bzw. Kernel):

- Vektor von Gewichten, visualisiert als Matrix in der Dimension des rezeptiven Felds
- Speichert das Muster bzw. Template
- Pixel des rezeptiven Felds werden mit dem Filter multipliziert (gewichtete Summe)
 - Bestimmung der Ähnlichkeit zwischen Eingabe und gespeichertem Template

Gewicht 1 →

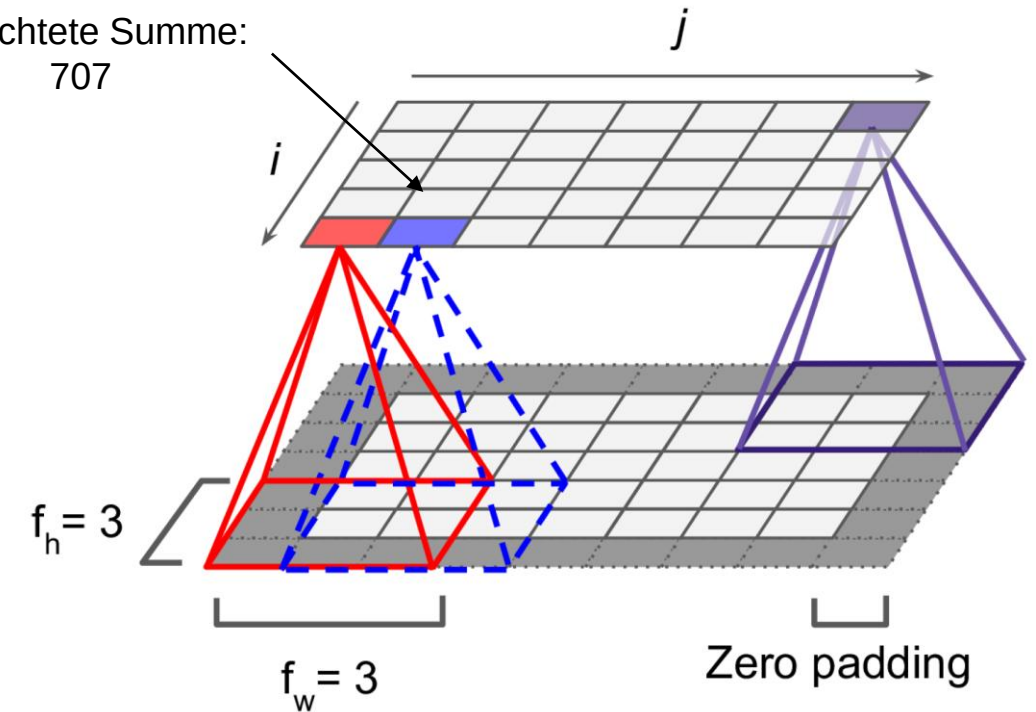
1	0	-1
1	0	-1
1	0	-1

Gewicht 4 →

Rezeptives Feld:

250	250	1
233	255	0
227	254	2

Gewichtete Summe:
707

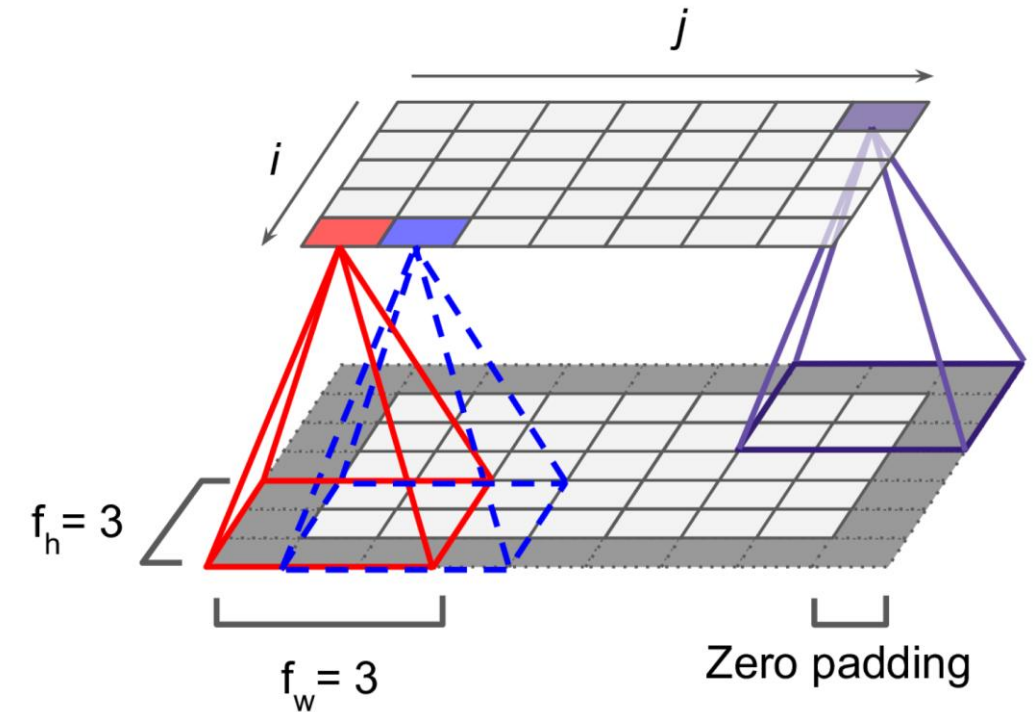


Convolutional Neural Network - CNN

Aufbau eines CNN

- Wh.:
Neuronen einer Schicht teilen sich den Kernel
-
-

1	0	-1
1	0	-1
1	0	-1



Convolutional Neural Network - CNN

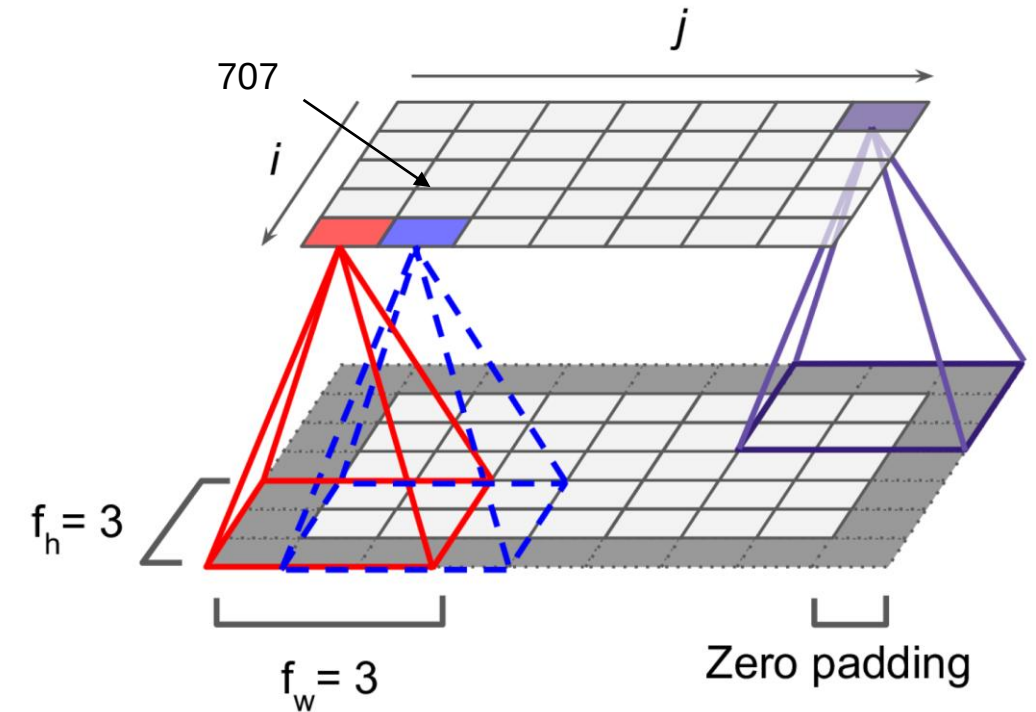
Aufbau eines CNN

- Wh.:
Neuronen einer Schicht teilen sich den Kernel
- Eingabe wird mit den Filtern gefaltet
 - Bestimmung der Ähnlichkeit zwischen Eingabe und gespeichertem Template

Eingabebild:

1	0	-1
1	0	-1
1	0	-1

250	250	1	210	180
233	255	0	255	200
227	254	2	234	254



Convolutional Neural Network - CNN

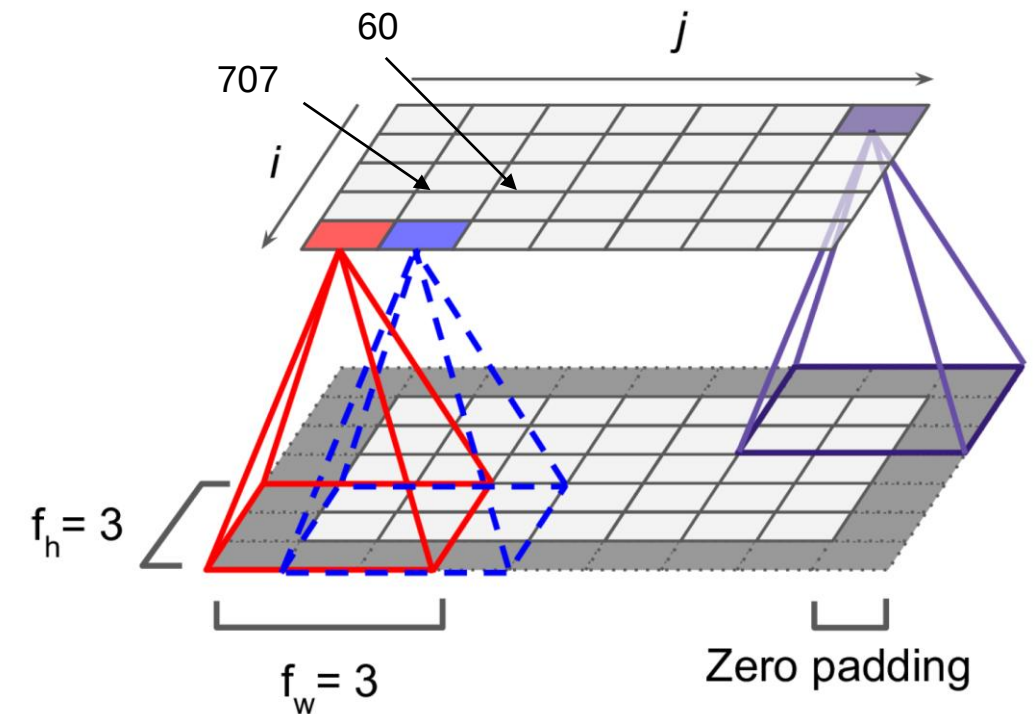
Aufbau eines CNN

- Wh.:
Neuronen einer Schicht teilen sich den Kernel
- Eingabe wird mit den Filtern gefaltet
 - Bestimmung der Ähnlichkeit zwischen Eingabe und gespeichertem Template

Eingabebild:

1	0	-1
1	0	-1
1	0	-1

250	250	1	210	180
233	255	0	255	200
227	254	2	234	254



Convolutional Neural Network - CNN

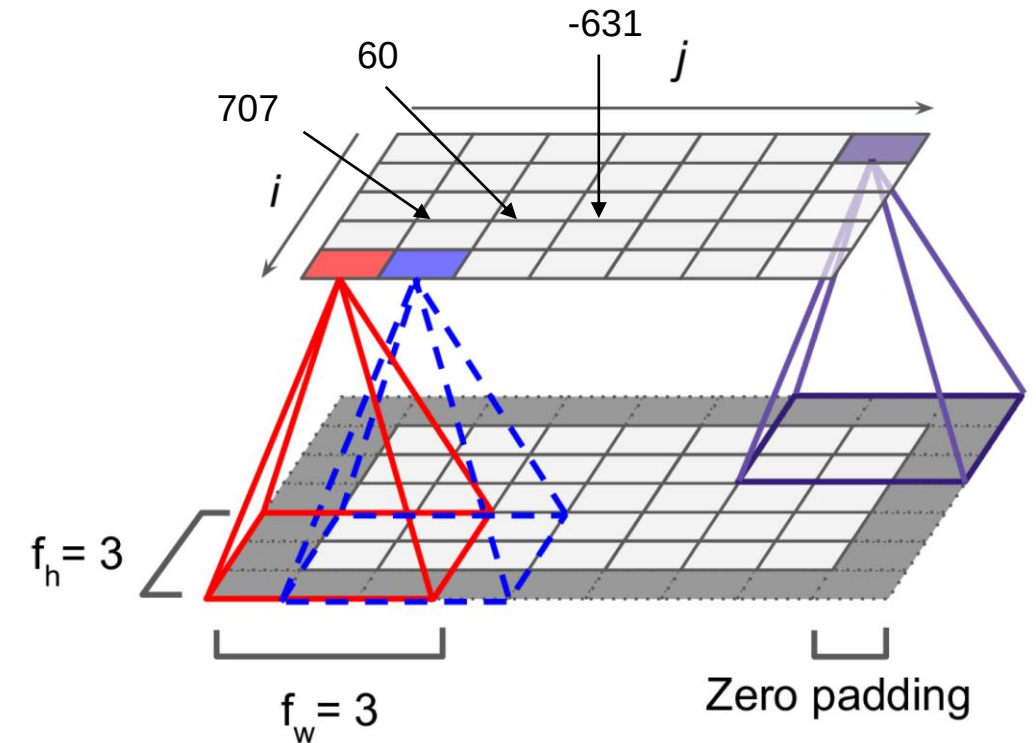
Aufbau eines CNN

- Wh.:
Neuronen einer Schicht teilen sich den Kernel
- Eingabe wird mit den Filtern gefaltet
 - Bestimmung der Ähnlichkeit zwischen Eingabe und gespeichertem Template

Eingabebild:

1	0	-1
1	0	-1
1	0	-1

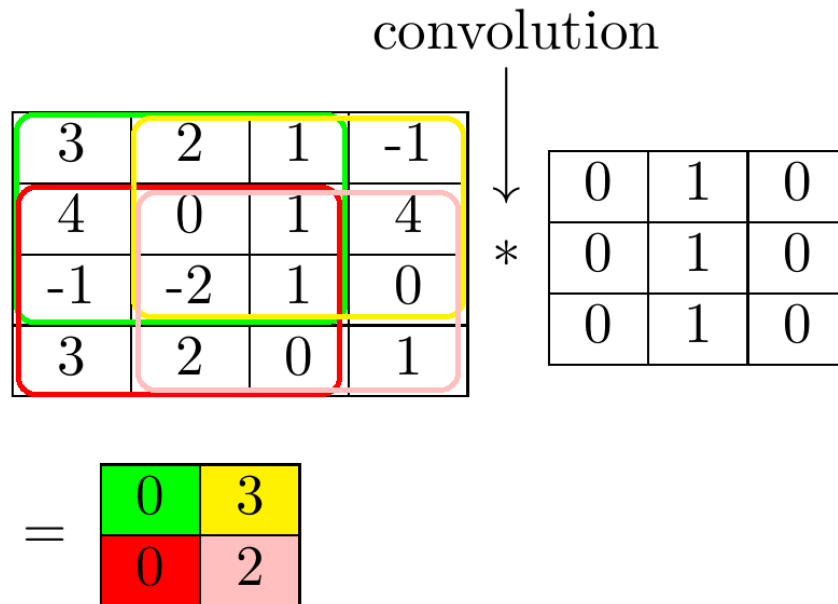
250	250	1	210	180
233	255	0	255	200
227	254	2	234	254



Convolutional Neural Network - CNN

Beispiel: Faltung mit einem Filter

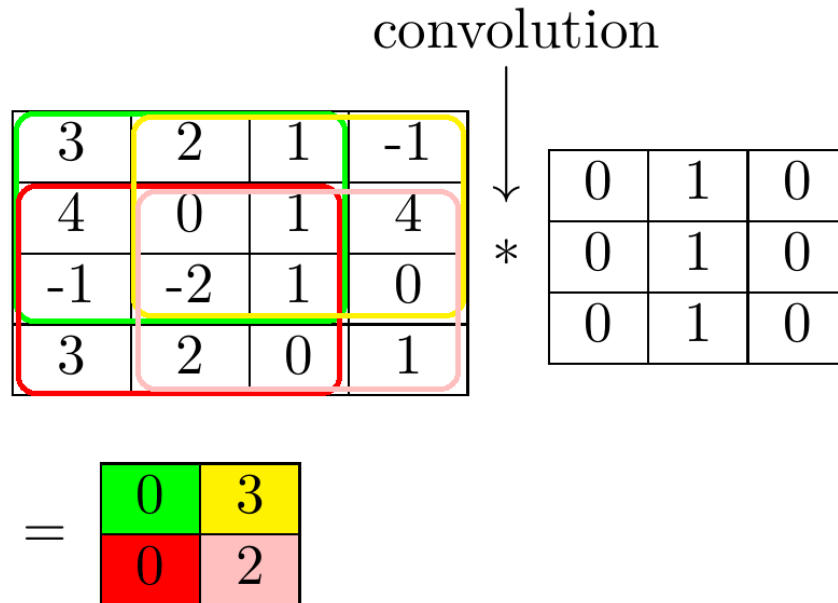
Beispiel: Anwendung eines Filters



Convolutional Neural Network - CNN

Beispiel: Faltung mit einem Filter

Beispiel: Anwendung eines Filters



Berechnung der **0**:

$$(3 \cdot 0 + 4 \cdot 0 + (-1) \cdot 0) +$$

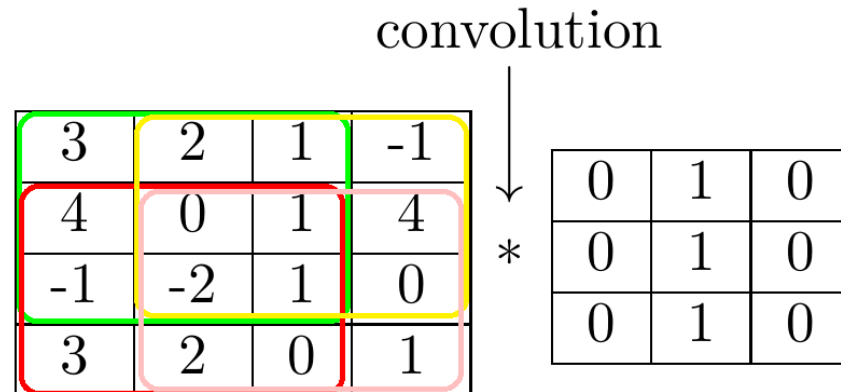
$$(2 \cdot 1 + 0 \cdot 1 + (-2) \cdot 1) +$$

$$(1 \cdot 0 + 1 \cdot 0 + 1 \cdot 0) = 0$$

Convolutional Neural Network - CNN

Beispiel: Faltung mit einem Filter

Beispiel: Anwendung eines Filters



Berechnung der **0**:

$$(3 \cdot 0 + 4 \cdot 0 + (-1) \cdot 0) +$$

$$(2 \cdot 1 + 0 \cdot 1 + (-2) \cdot 1) +$$

$$(1 \cdot 0 + 1 \cdot 0 + 1 \cdot 0) = 0$$

=

0	3
0	2

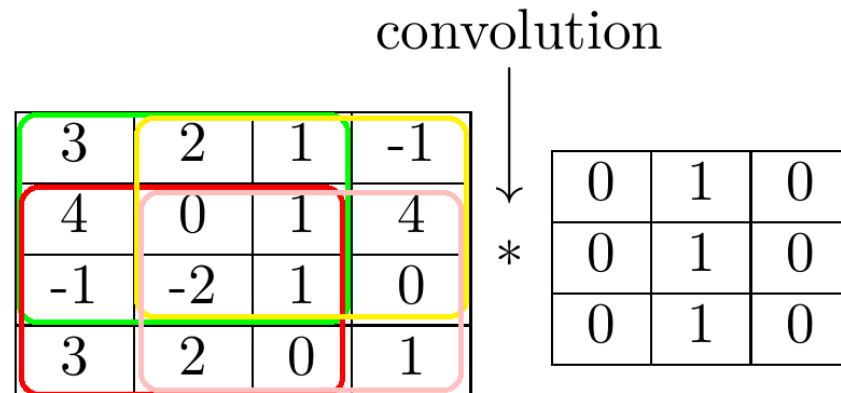
← Gewichtete Summe von Neuron 2

← Gewichtete Summe von Neuron 4

Convolutional Neural Network - CNN

Beispiel: Faltung mit einem Filter

Beispiel: Anwendung eines Filters



Berechnung der **0**:

$$(3 \cdot 0 + 4 \cdot 0 + (-1) \cdot 0) +$$

$$(2 \cdot 1 + 0 \cdot 1 + (-2) \cdot 1) +$$

$$(1 \cdot 0 + 1 \cdot 0 + 1 \cdot 0) = 0$$

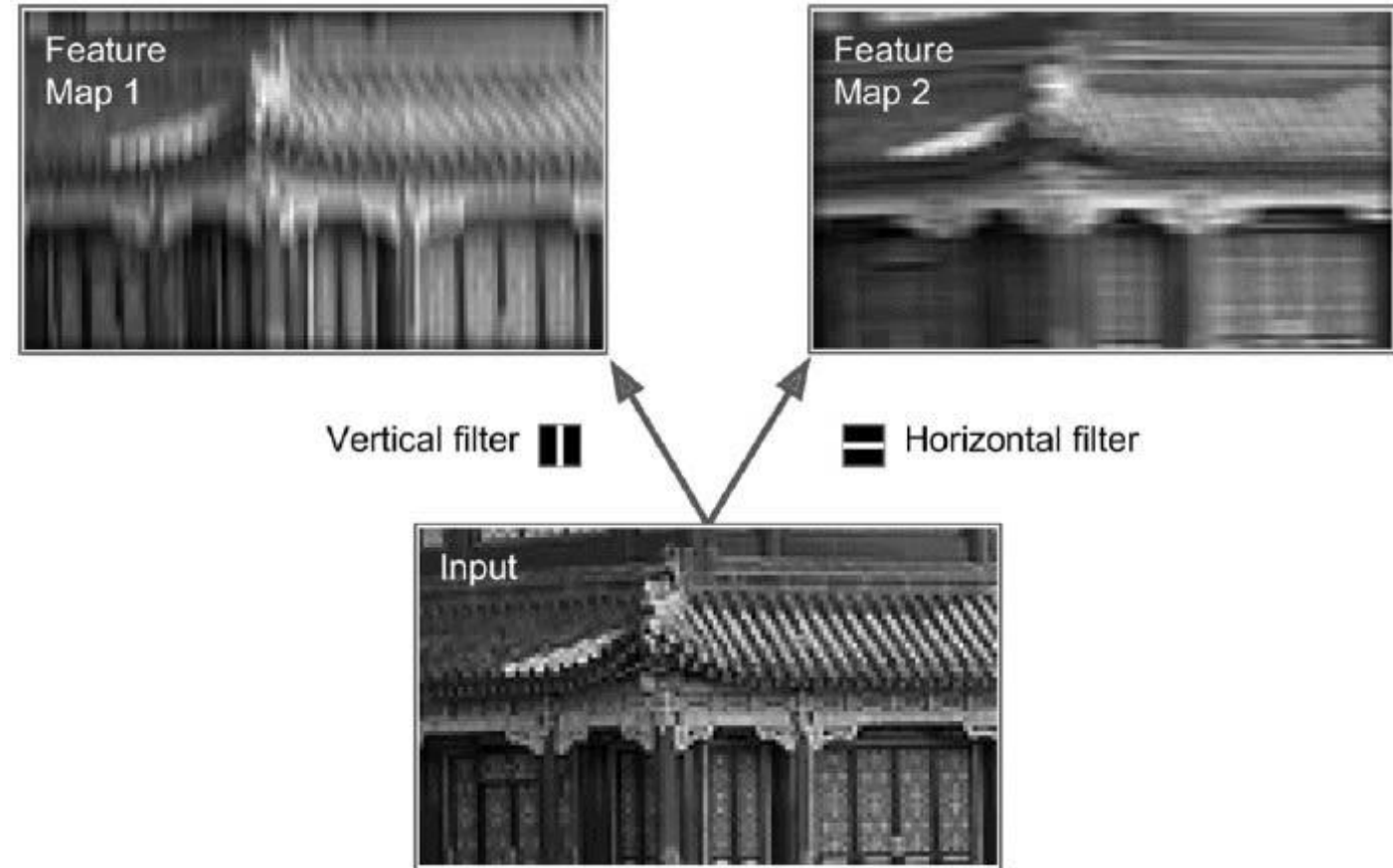


→ Berechnung der gewichteten Summe aller Neuronen einer Schicht ist identisch mit der Faltung der Eingabe mit der Filtermatrix der Schicht

Convolutional Neural Network - CNN

Filter

- Filter heißen so, weil sie nur bestimmte Bildinformationen durchlassen
- Vertikale Filter lassen nur vertikale Kanten sichtbar zurück
- Horizontale Filter heben horizontale Kanten hervor

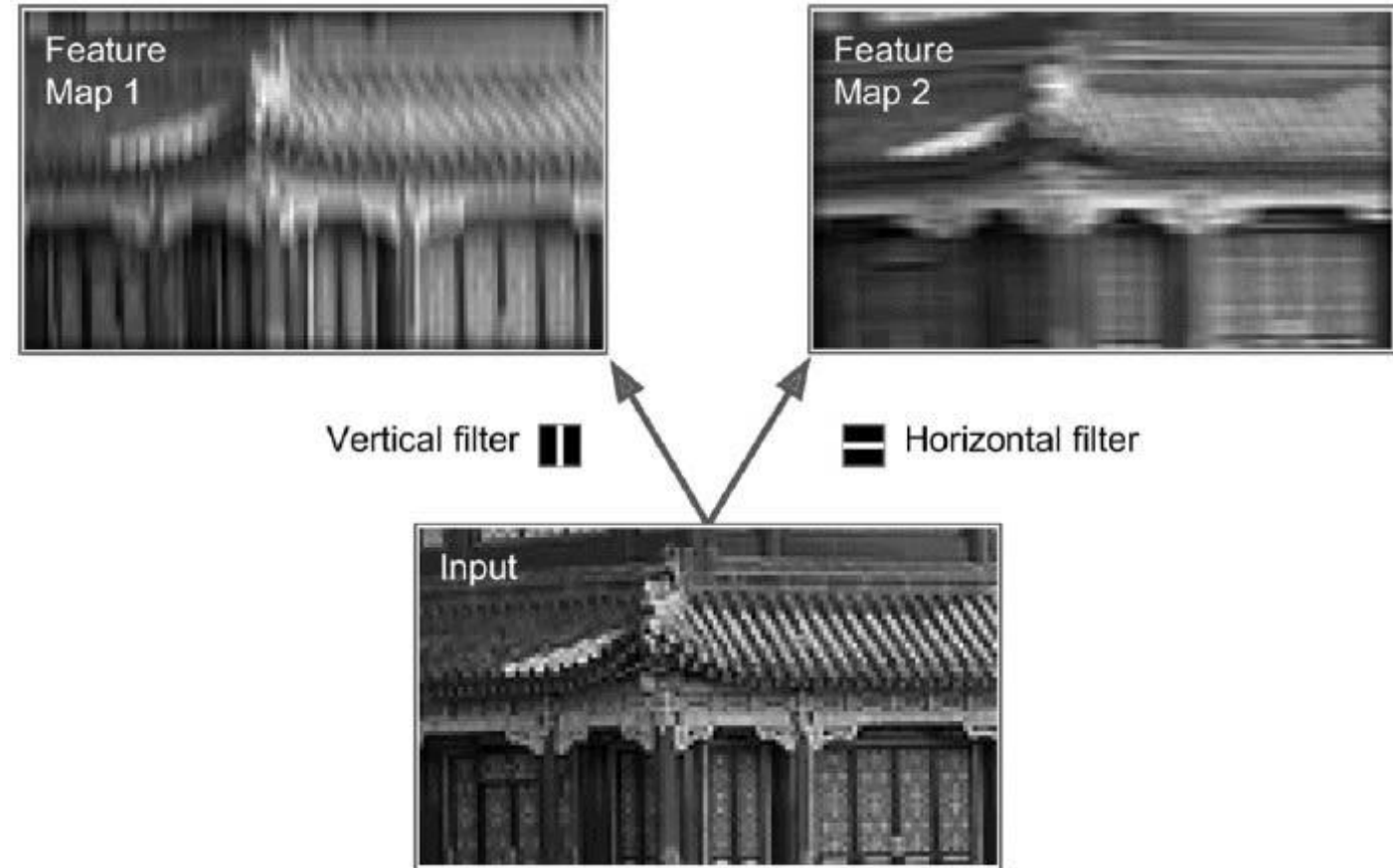


Convolutional Neural Network - CNN

Filter

- Filter heißen so, weil sie nur bestimmte Bildinformationen durchlassen
- Vertikale Filter lassen nur vertikale Kanten sichtbar zurück
- Horizontale Filter heben horizontale Kanten hervor

1	1	1
0	0	0
-1	-1	-1



Convolutional Neural Network - CNN

Übung: Filter

- Berechnen Sie:

Eingabebild:

250	250	1	210	180
233	255	0	255	200
227	254	2	234	254

*

1	1	1
0	0	0
-1	-1	-1

=

Convolutional Neural Network - CNN

Übung: Filter

- Berechnen Sie:

Eingabebild:

250	250	1	210	180
233	255	0	255	200
227	254	2	234	254

*

1	1	1
0	0	0
-1	-1	-1

=

18	-29	-99
----	-----	-----

Convolutional Neural Network - CNN

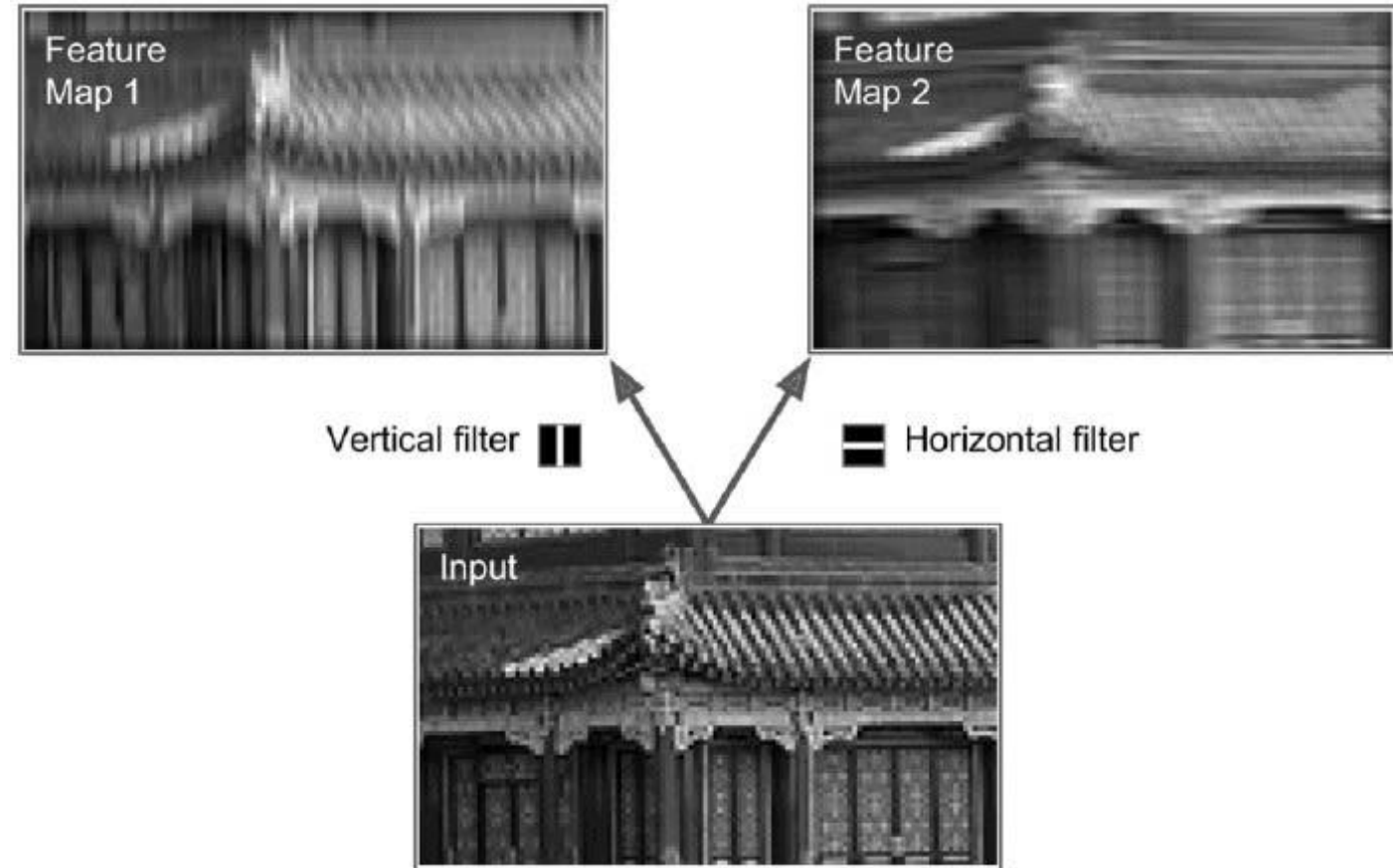
Leitfragen:

- Wie erkennt ein CNN unterschiedliche Muster (bspw. horizontale, schräge und vertikale Kanten)?
- Wie erkennt ein CNN komplexere Merkmale (bspw. Kreuzungen)?

Convolutional Neural Network - CNN

Feature Map

- Feature Map = Ausgangsbild einer Convolutional Schicht, die als Eingangsbild der nächsten Convolutional Schicht genutzt wird
- Da wir unterschiedliche Muster detektieren möchten, benötigen wir mehrere Filter pro Schicht

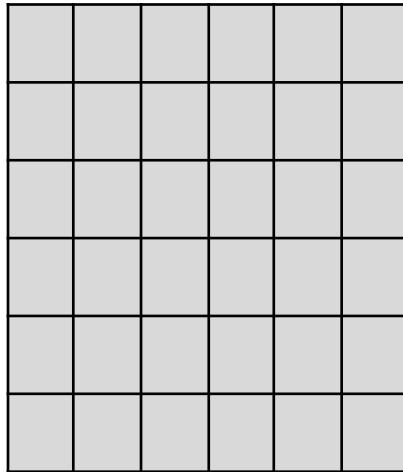


Convolutional Neural Network - CNN

Feature Map

Hier im Beispiel:
Stride = 1
Kein Padding

Eingangsbild
(Graustufenbild)



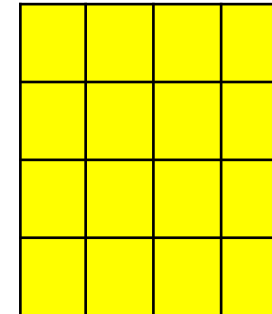
6x6 Pixel

2 Filter (vertikale und
horizontale Kanten)

*

1	0	-1
1	0	-1
1	0	-1

=

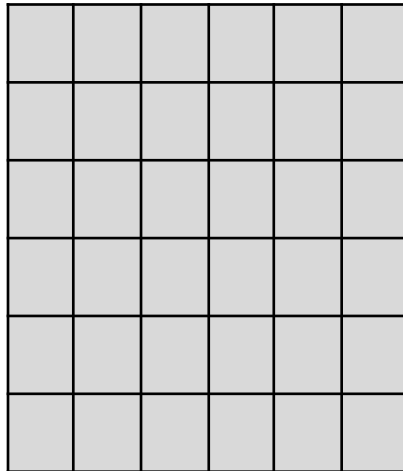


Convolutional Neural Network - CNN

Feature Map

Hier im Beispiel:
Stride = 1
Kein Padding

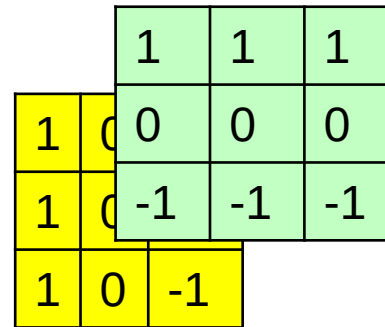
Eingangsbild
(Graustufenbild)



6x6 Pixel

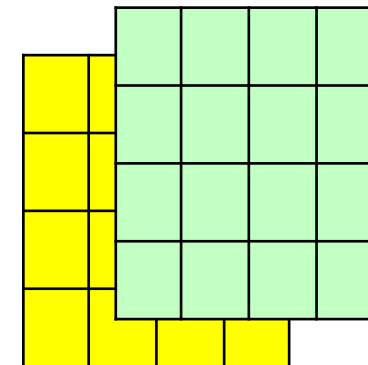
2 Filter (vertikale und
horizontale Kanten)

*



=

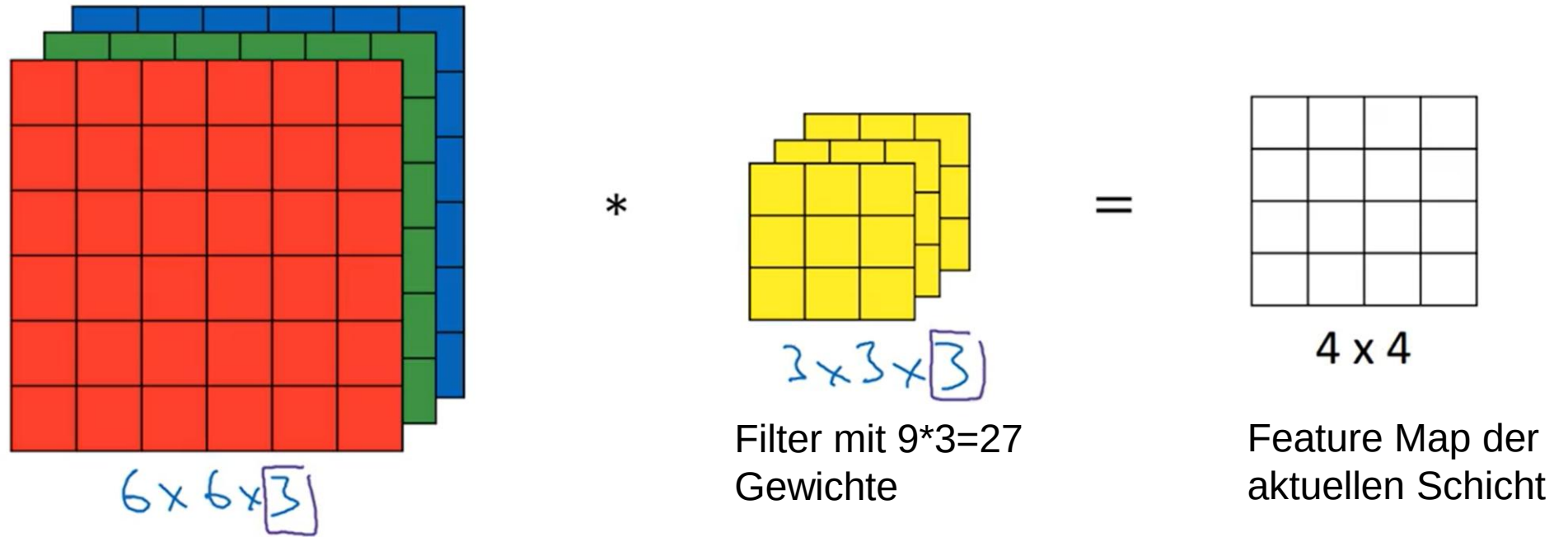
2 Feature Maps



4x4x2 Pixel

Convolutional Neural Network - CNN

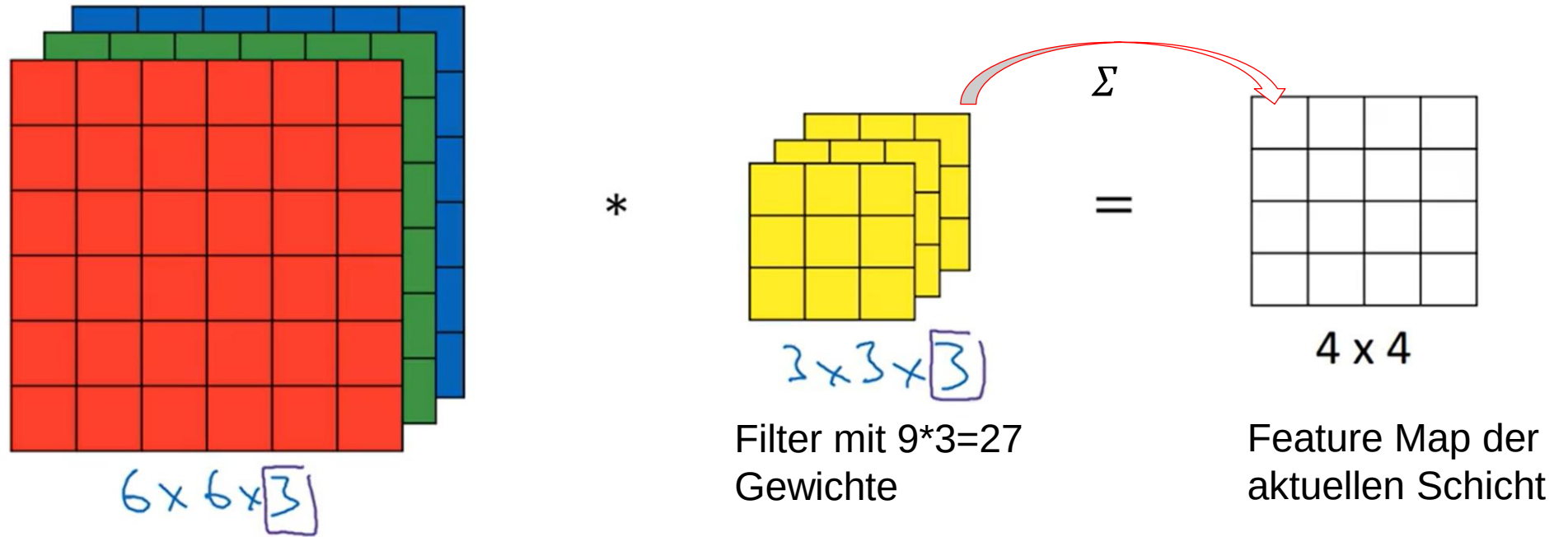
Multiple Feature Maps



Filter hat so viele Kanäle wie es Feature Maps gibt, die in die Schicht hineingehen.

Convolutional Neural Network - CNN

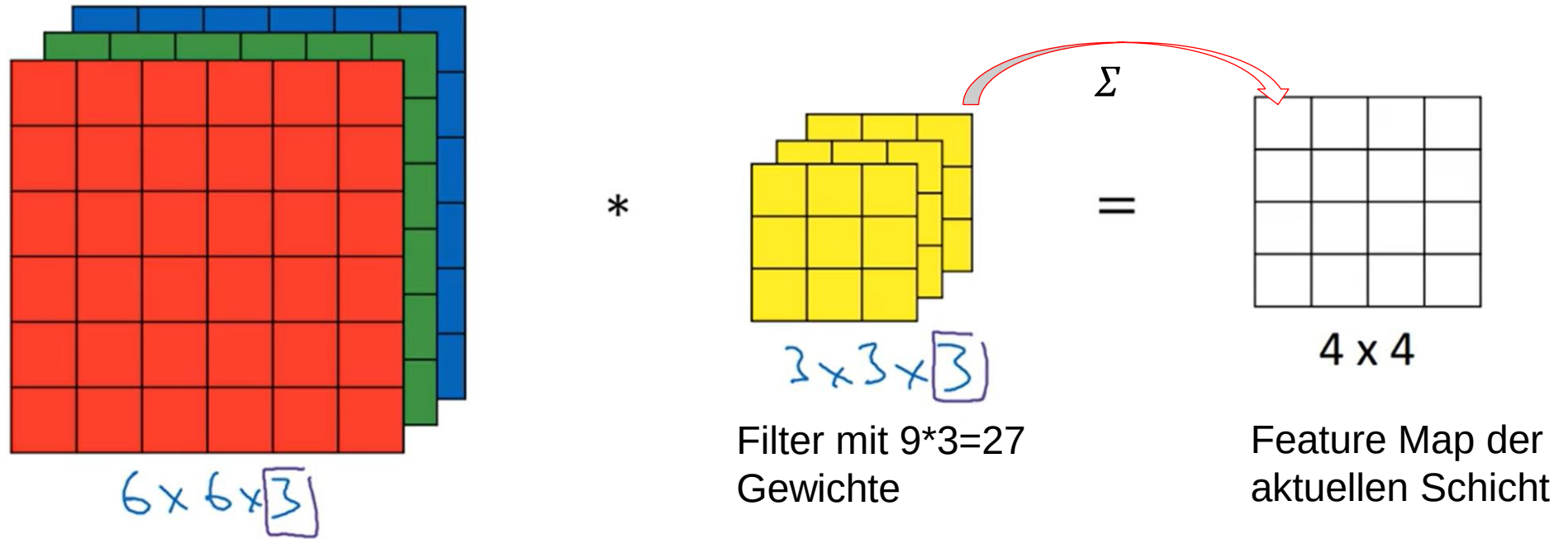
Multiple Feature Maps



Filter hat so viele Kanäle wie es Feature Maps gibt, die in die Schicht hineingehen.

Convolutional Neural Network - CNN

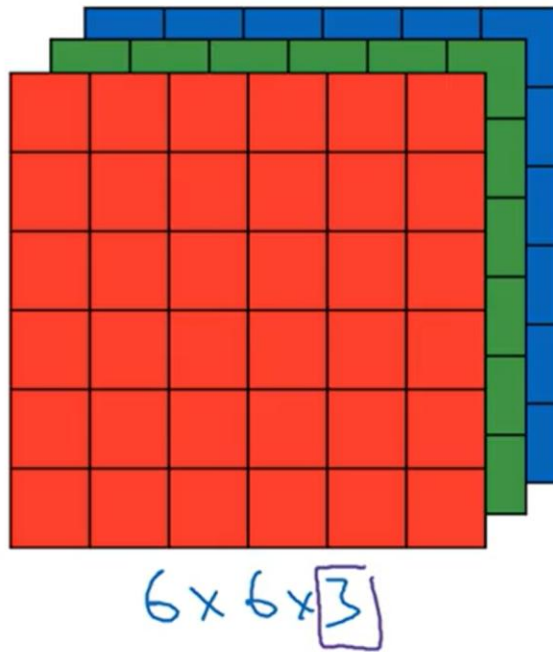
Multiple Feature Maps



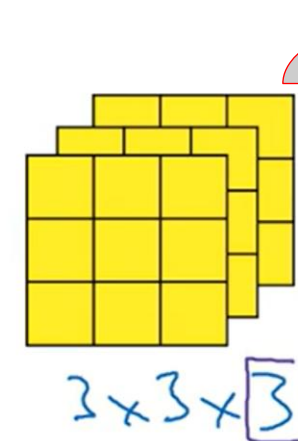
Filter hat so viele Kanäle wie es Feature Maps gibt, die in die Schicht hineingehen.
 Kanal 1 des Filters könnte nach horizontalen Kanten in Feature Map 1 schauen, Kanal 2 nach vertikalen Kanten in Feature Map 2, ... dadurch können komplexe Features erkannt werden

Convolutional Neural Network - CNN

Multiple Feature Maps



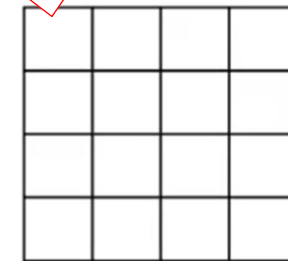
*



Filter mit $9 \times 3 = 27$
Gewichte

Σ

=



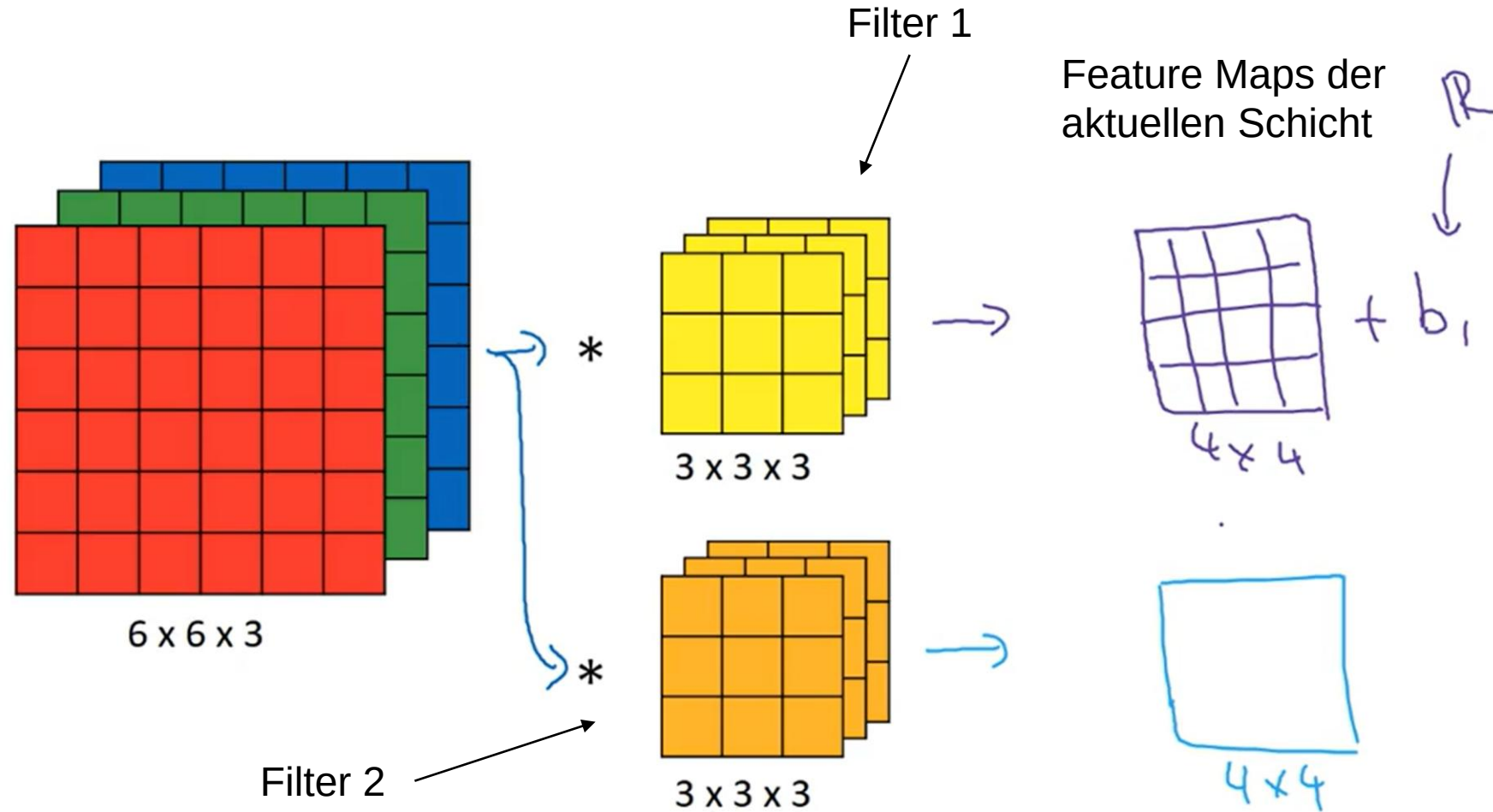
4 x 4

Feature Map der
aktuellen Schicht

Filter hat so viele Kanäle wie es Feature Maps gibt, die in die Schicht hineingehen.
Kanal 1 des Filters könnte nach horizontalen Kanten in Feature Map 1 schauen, Kanal 2 nach vertikalen Kanten in Feature Map 2, ... dadurch können komplexe Features erkannt werden

Convolutional Neural Network - CNN

Multiple Feature Maps



Convolutional Neural Network - CNN

Beispiel eines trivialen CNNs

[04_synthetischer_datensatz_cnn.ipynb](#)

```
model = keras.Sequential([
    keras.layers.Conv2D(filters=2, kernel_size=(3, 3), activation='linear', input_shape=(24, 24, 1)),
    keras.layers.Flatten(),
    keras.layers.Dense(2, activation='softmax')
])
```

- Zwei Klassen:



Convolutional Neural Network - CNN

Beispiel eines trivialen CNNs

[04_synthetischer_datensatz_cnn.ipynb](#)

Feature Maps der ersten Convolutional Schicht:

Hier sind beide Gewichtsmatrizen identisch zu horizontaler/vertikaler Kantenfilter (Prewitt Operator)

Aktivierungen (Vertikal) - Layer 1

Layer 1, Filter 1



Layer 1, Filter 2



Convolutional Neural Network - CNN

[04_synthetischer_datensatz_cnn.ipynb](#)

Beispiel eines trivialen CNNs

Feature Maps der ersten Convolutional Schicht:

Hier sind beide Gewichtsmatrizen identisch zu horizontaler/vertikaler Kantenfilter (Prewitt Operator)

Aktivierungen (Vertikal) - Layer 1

Layer 1, Filter 1



Layer 1, Filter 2



Aktivierungen (Horizontal) - Layer 1

Layer 1, Filter 1



Layer 1, Filter 2



Convolutional Neural Network - CNN

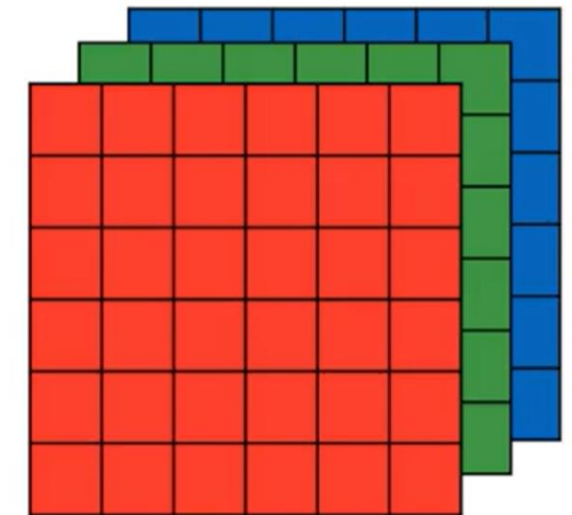
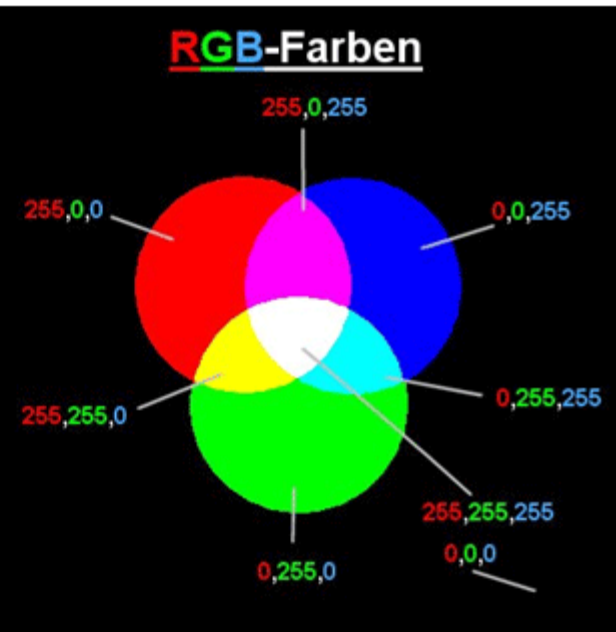
Leitfragen:

- Wie verarbeitet ein CNN Farbbilder?

Farbbilder

- Was ist der RGB Farbraum? (Additive Farbmischung)

Farbnamen	Hex-Wert	R	G	B
black	#000000	0	0	0
blue	#0000FF	0	0	255
lime	#00FF00	0	255	0
cyan	#00FFFF	0	255	255
red	#FF0000	255	0	0
magenta	#FF00FF	255	0	255
yellow	#FFFF00	255	255	0
white	#FFFFFF	255	255	255



6 x 6 x 3

- D.h., Farbbilder werden durch die drei Kanäle rot, grün, blau dargestellt und werden als RGB-Bilder bezeichnet

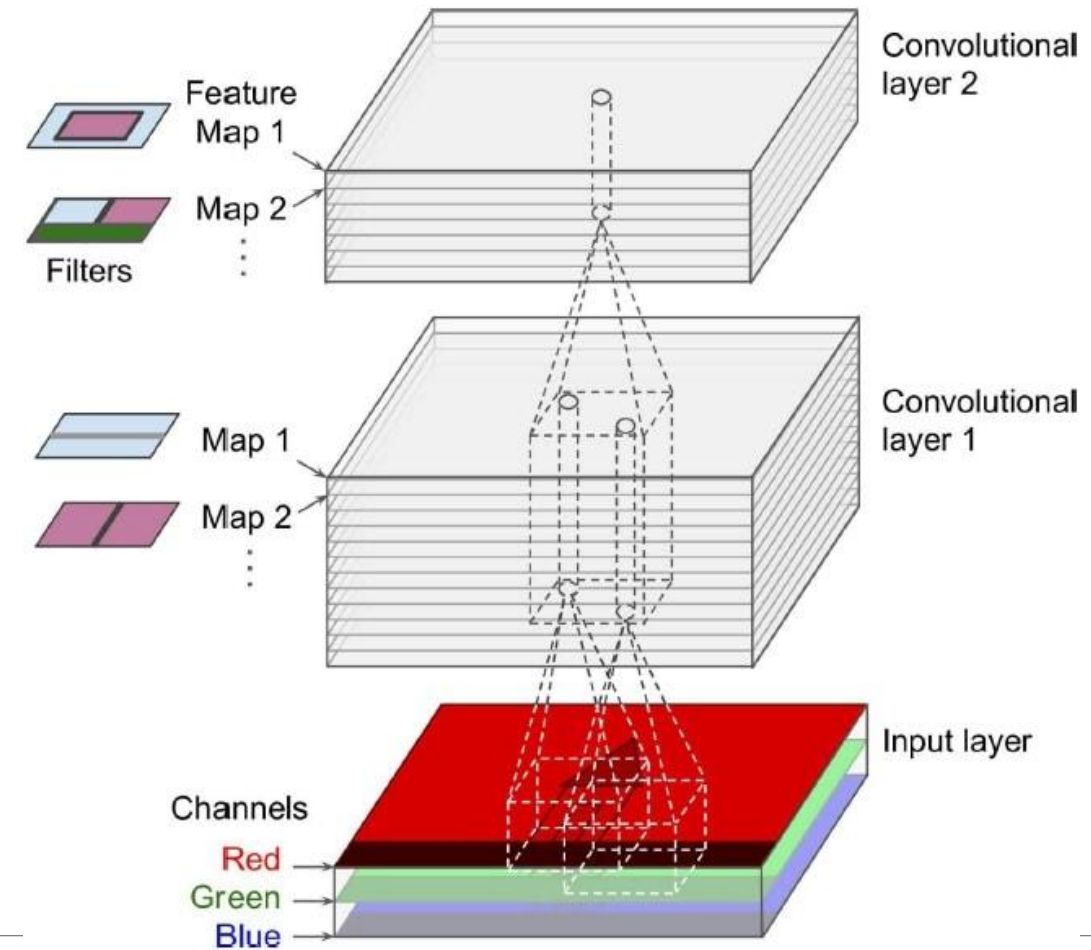
Convolutional Neural Network - CNN

Multiple Feature Maps

Eingabe

- Farbbild (3 Kanäle: RGB), Graustufenbild (1 Kanal)

-
-
-



Convolutional Neural Network - CNN

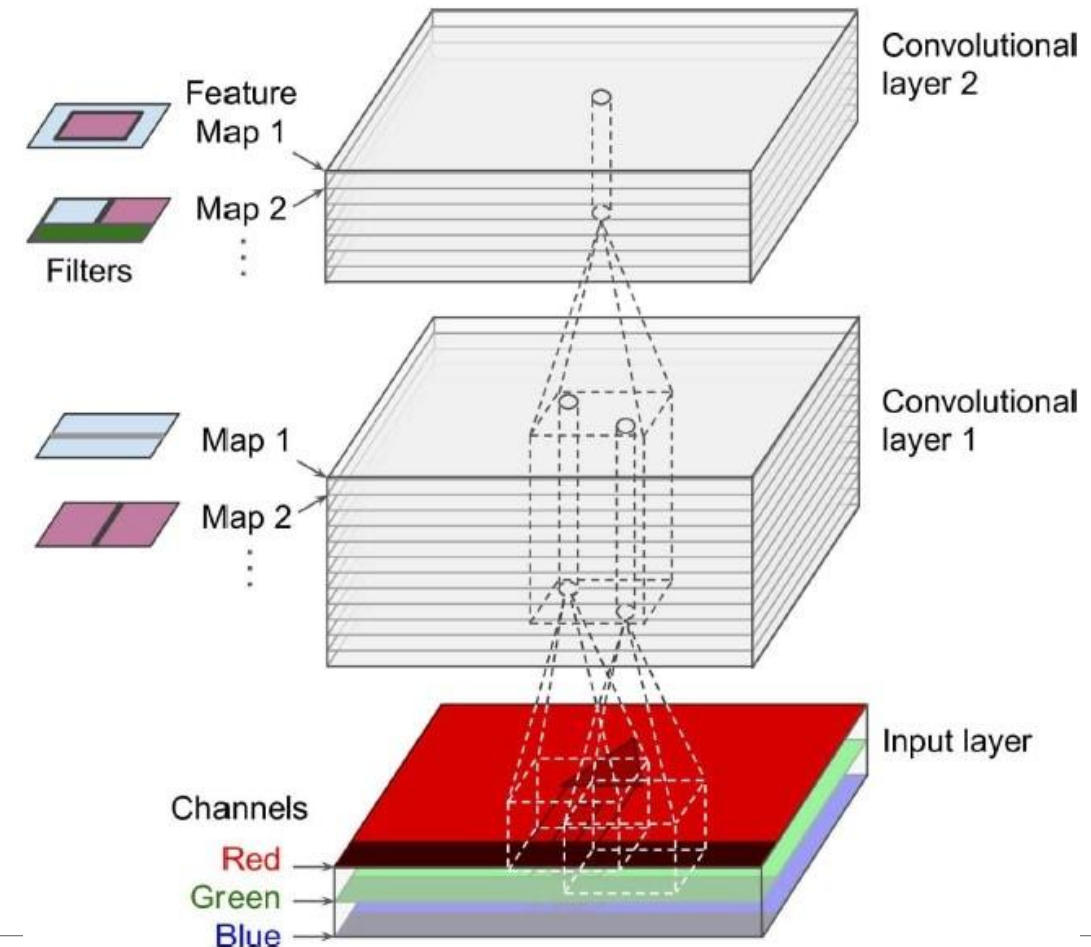
Multiple Feature Maps

Eingabe

- Farbbild (3 Kanäle: RGB), Graustufenbild (1 Kanal)

Neuronen ...

- ... einer Feature Map besitzen gleiche Gewichte
- ... verschiedener Feature Maps besitzen unterschiedliche Gewichte
-



Convolutional Neural Network - CNN

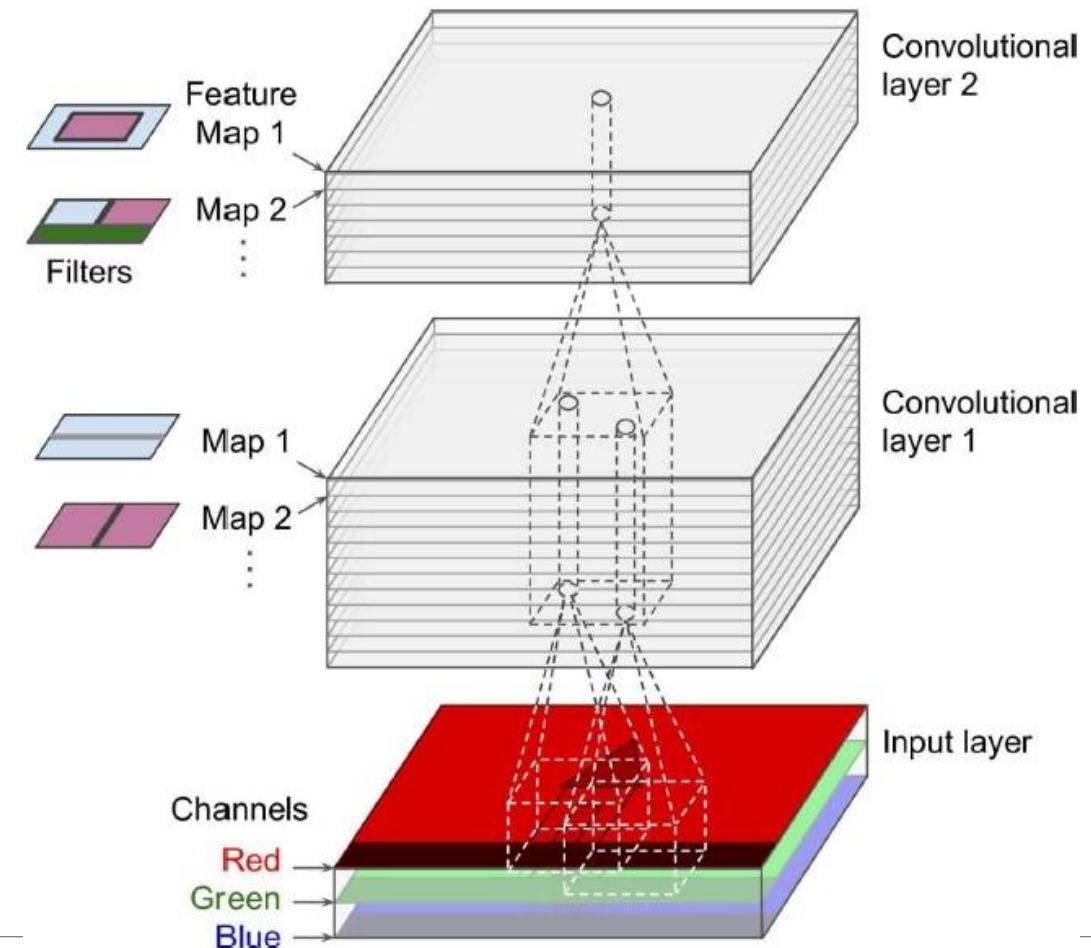
Multiple Feature Maps

Eingabe

- Farbbild (3 Kanäle: RGB), Graustufenbild (1 Kanal)

Neuronen ...

- ... einer Feature Map besitzen gleiche Gewichte
- ... verschiedener Feature Maps besitzen unterschiedliche Gewichte
- Rezeptives Feld durchdringt alle Feature Maps der vorherigen Schicht

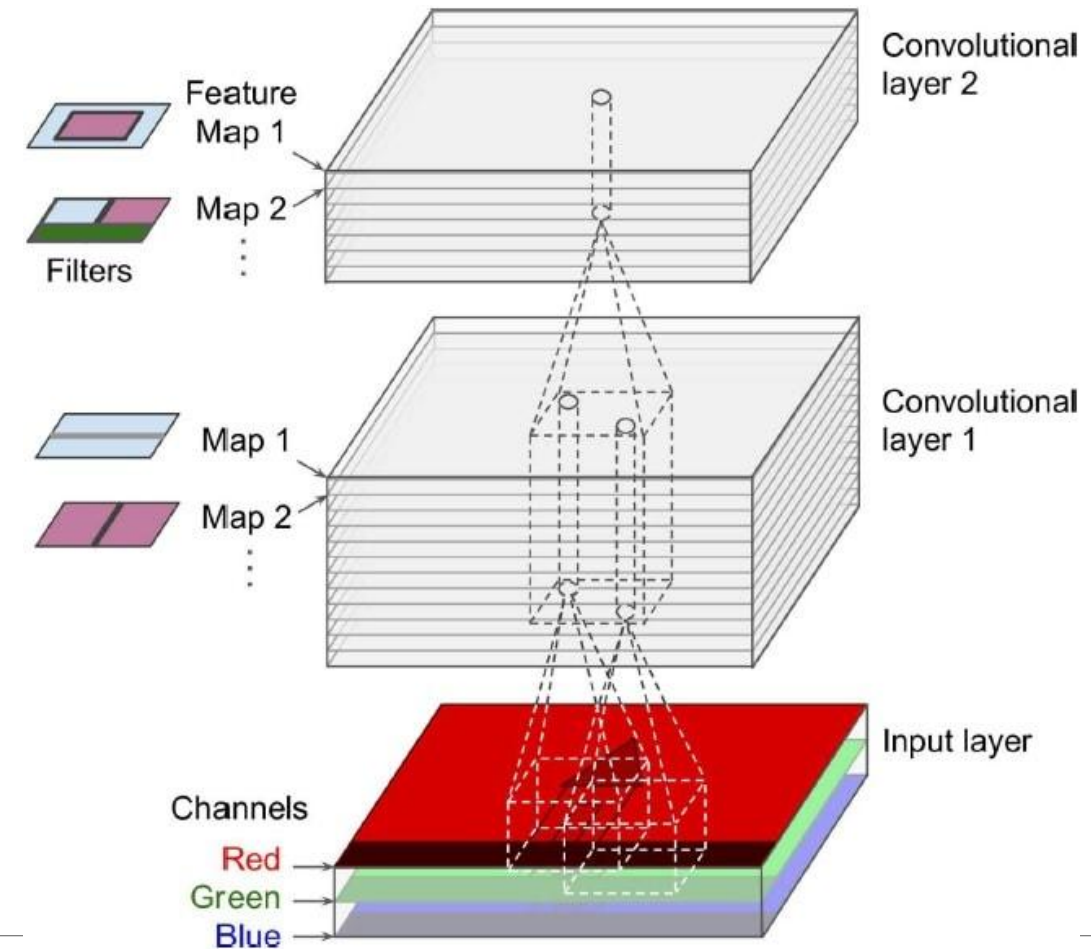


Convolutional Neural Network - CNN

Multiple Feature Maps

Beispielrechnung:

- Annahmen:
 - jedes Neuron hat ein rezeptives Feld von 3x3 Pixel
 - Layer 1 hat 10 Filter (also 10 Feature Maps)



Convolutional Neural Network - CNN

Multiple Feature Maps

Beispielrechnung:

- Annahmen:
 - jedes Neuron hat ein rezeptives Feld von 3x3 Pixel
 - Layer 1 hat 10 Filter (also 10 Feature Maps)

- Anzahl Gewichte eines Neurons in Layer 1:

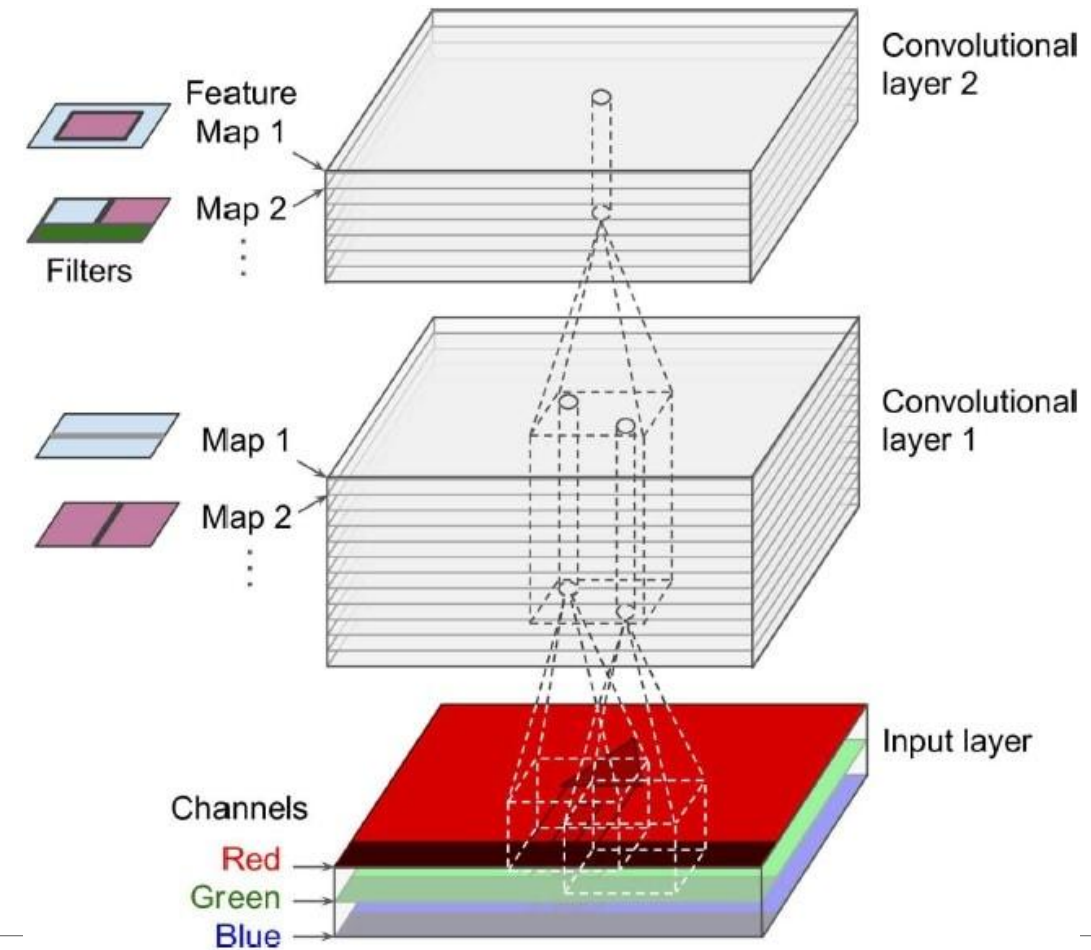
■

■

■

■

■



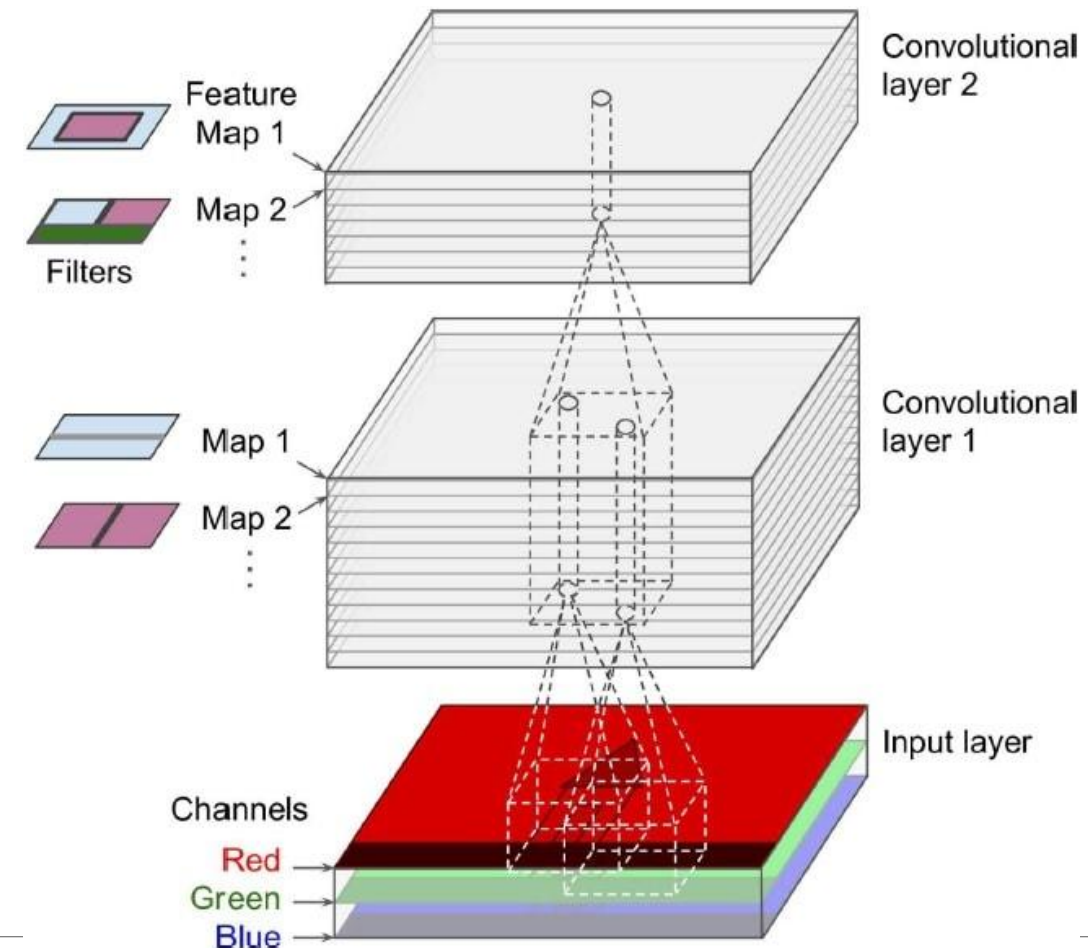
Convolutional Neural Network - CNN

Multiple Feature Maps

Beispielrechnung:

- Annahmen:
 - jedes Neuron hat ein rezeptives Feld von 3x3 Pixel
 - Layer 1 hat 10 Filter (also 10 Feature Maps)
- Anzahl Gewichte eines Neurons in Layer 1:
 - $3*3*3+1 = 28$

-
-
-
-

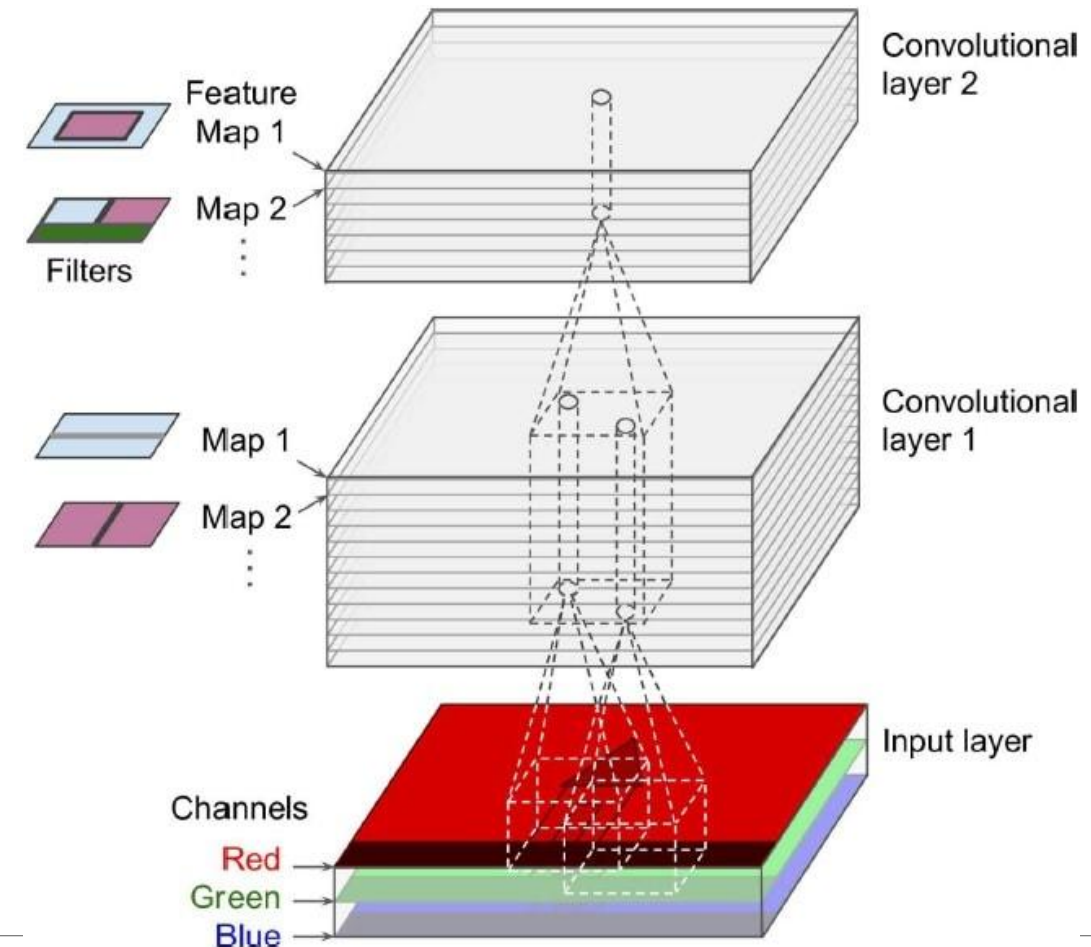


Convolutional Neural Network - CNN

Multiple Feature Maps

Beispielrechnung:

- Annahmen:
 - jedes Neuron hat ein rezeptives Feld von 3x3 Pixel
 - Layer 1 hat 10 Filter (also 10 Feature Maps)
- Anzahl Gewichte eines Neurons in Layer 1:
 - $3 \times 3 \times 3 + 1 = 28$
- Anzahl Gewichte eines Neurons in Layer 2:
 -

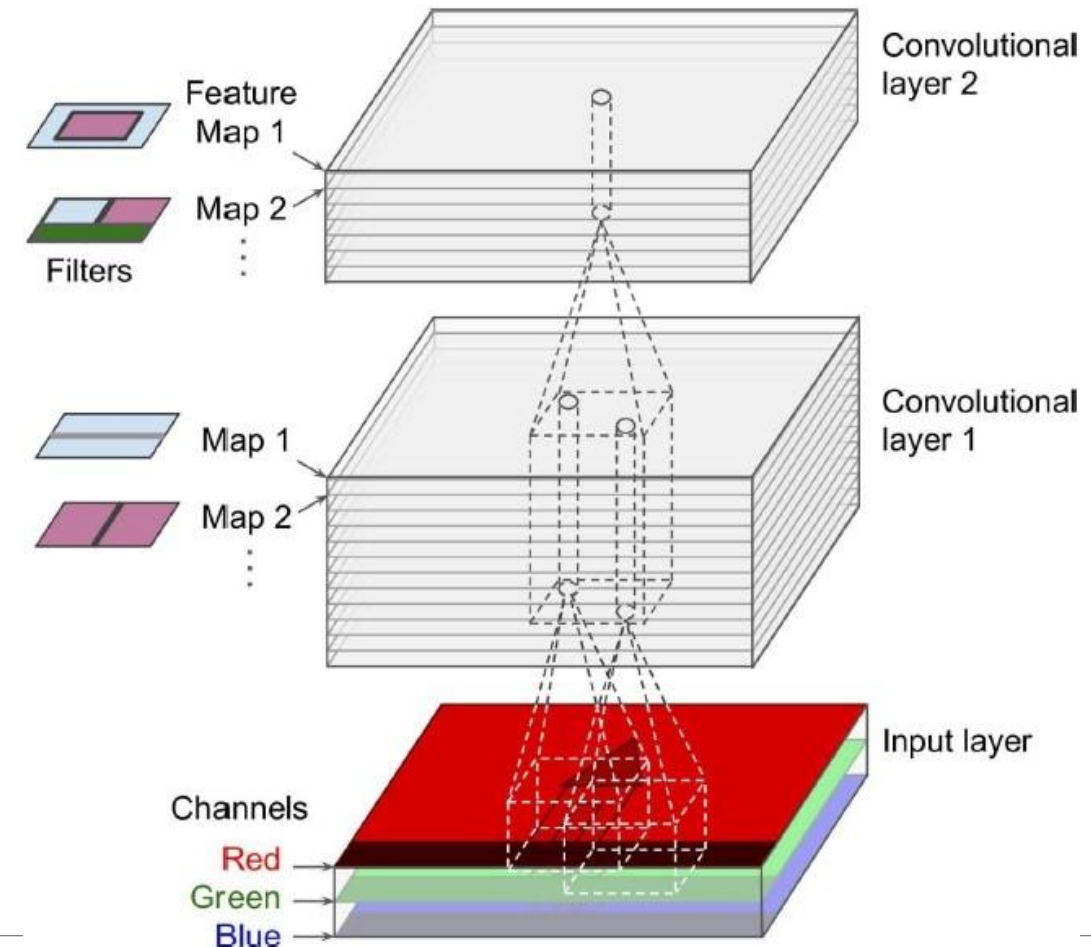


Convolutional Neural Network - CNN

Multiple Feature Maps

Beispielrechnung:

- Annahmen:
 - jedes Neuron hat ein rezeptives Feld von 3x3 Pixel
 - Layer 1 hat 10 Filter (also 10 Feature Maps)
- Anzahl Gewichte eines Neurons in Layer 1:
 - $3*3*3+1 = 28$
- Anzahl Gewichte eines Neurons in Layer 2:
 - $3*3*10+1 = 91$

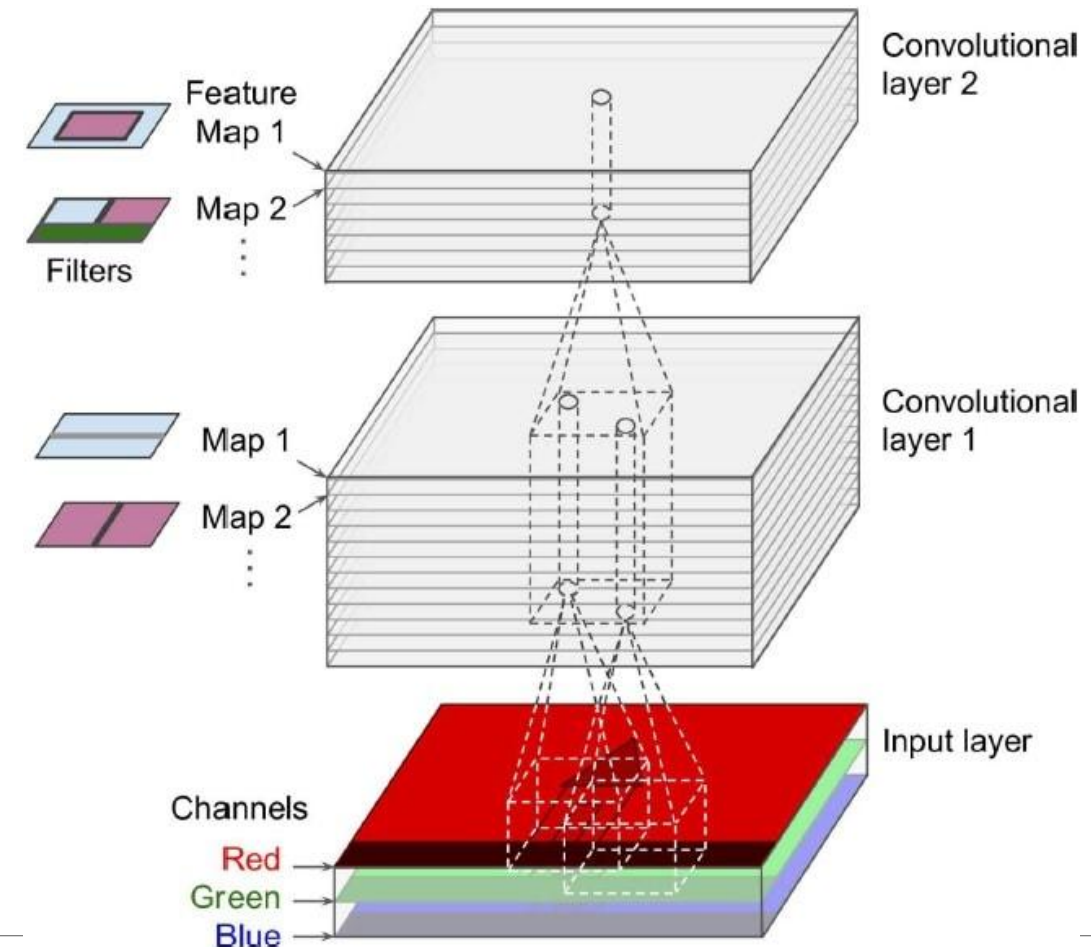


Convolutional Neural Network - CNN

Multiple Feature Maps

Beispielrechnung:

- Annahmen:
 - jedes Neuron hat ein rezeptives Feld von 3x3 Pixel
 - Layer 1 hat 10 Filter (also 10 Feature Maps)
- Anzahl Gewichte eines Neurons in Layer 1:
 - $3*3*3+1 = 28$
- Anzahl Gewichte eines Neurons in Layer 2:
 - $3*3*10+1 = 91$
- # Gewichte in Layer 1 über alle Feature Maps:
 -

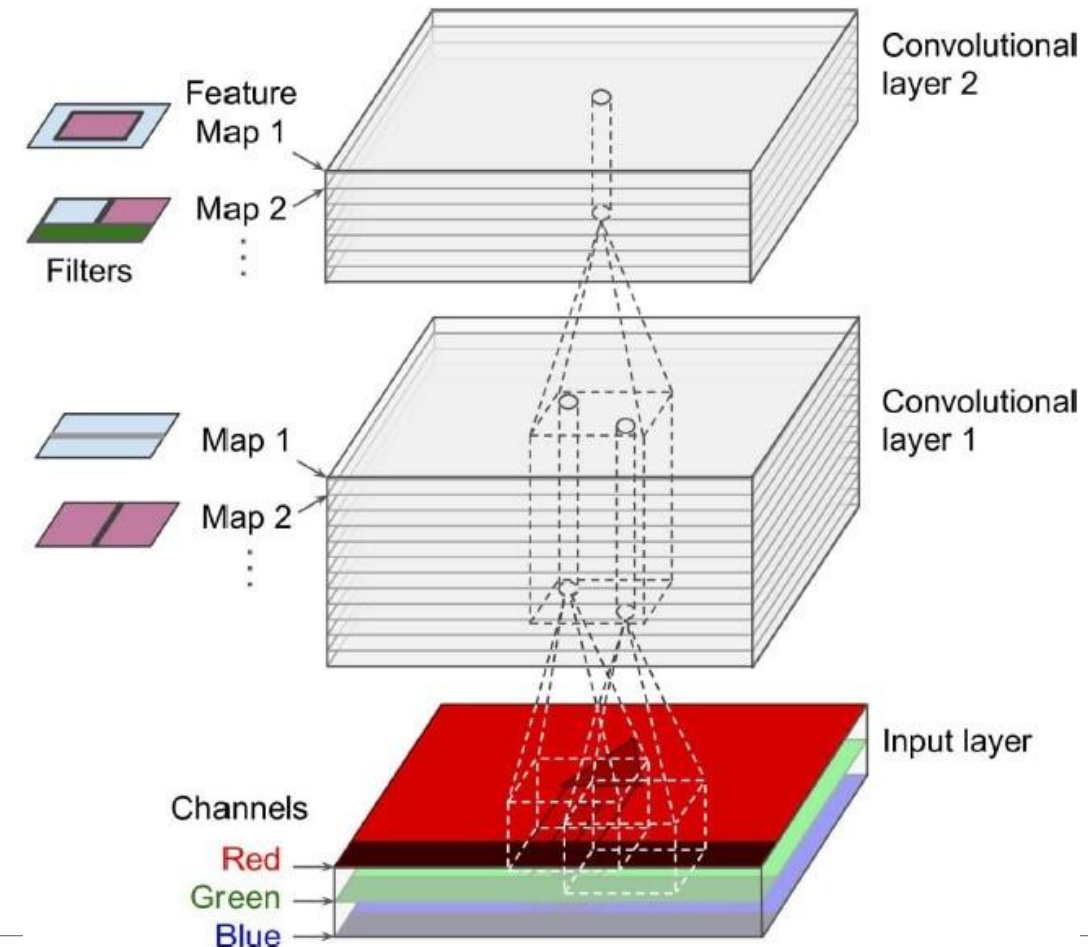


Convolutional Neural Network - CNN

Multiple Feature Maps

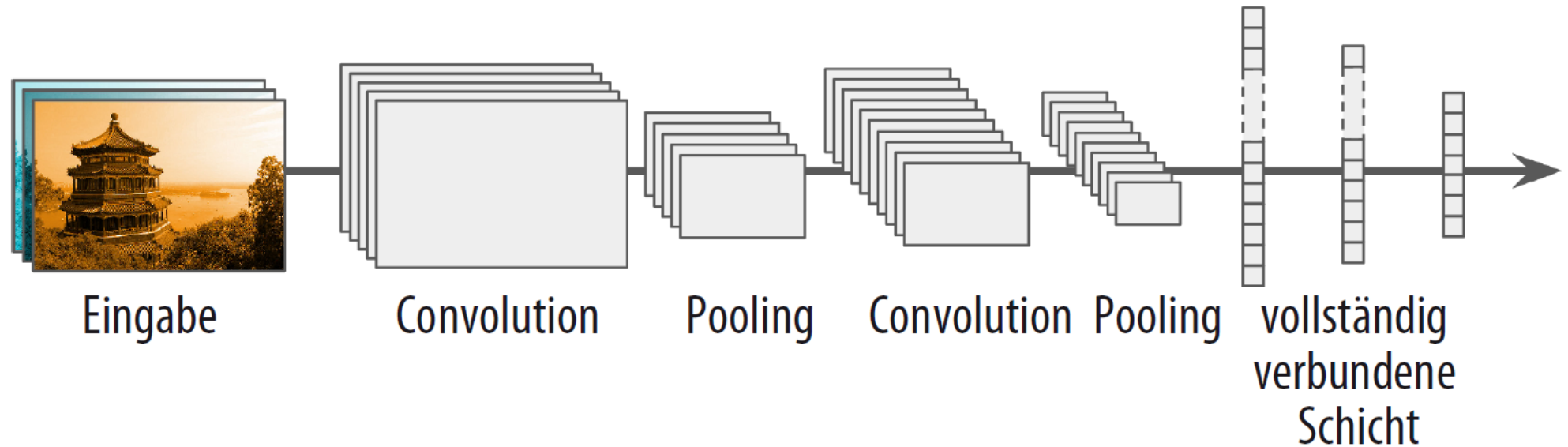
Beispielrechnung:

- Annahmen:
 - jedes Neuron hat ein rezeptives Feld von 3x3 Pixel
 - Layer 1 hat 10 Filter (also 10 Feature Maps)
- Anzahl Gewichte eines Neurons in Layer 1:
 - $3*3*3+1 = 28$
- Anzahl Gewichte eines Neurons in Layer 2:
 - $3*3*10+1 = 91$
- # Gewichte in Layer 1 über alle Feature Maps:
 - $28*10 = 280$



Convolutional Neural Network - CNN

Typische Architektur

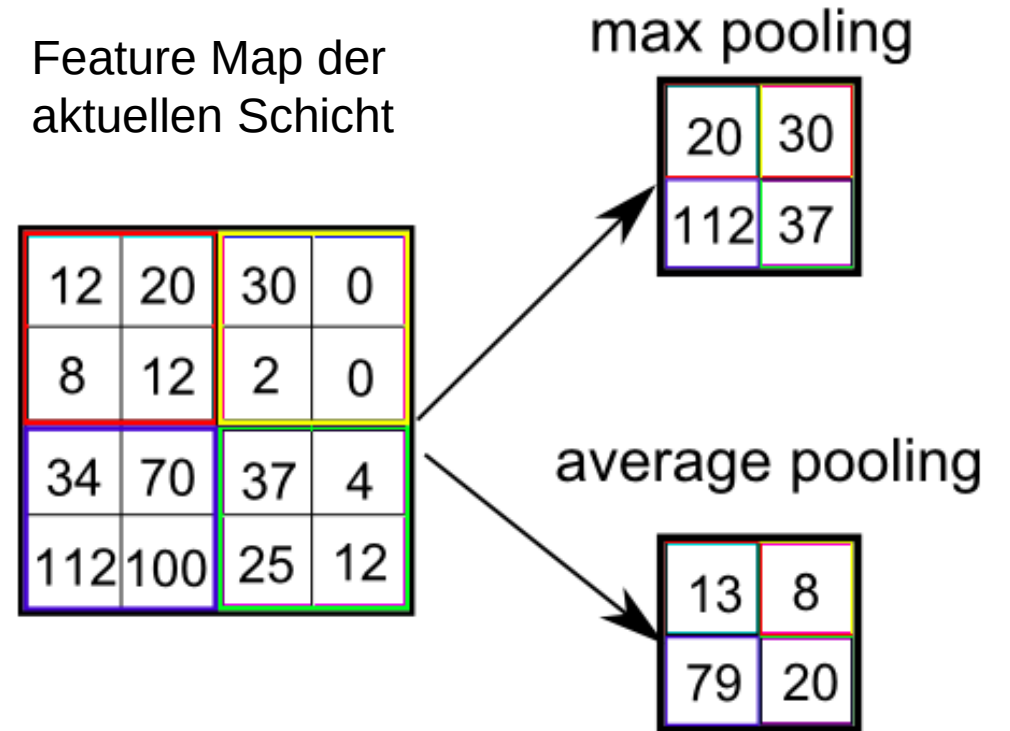


Convolutional Neural Network - CNN

Pooling-Schicht

Pooling-Schicht

- Aggregiert die Ausgabe einer Convolutional Schicht (nach der Aktivierungsfunktion)
 - Max oder Mean
 -
 -
 -
 -
 -
 -

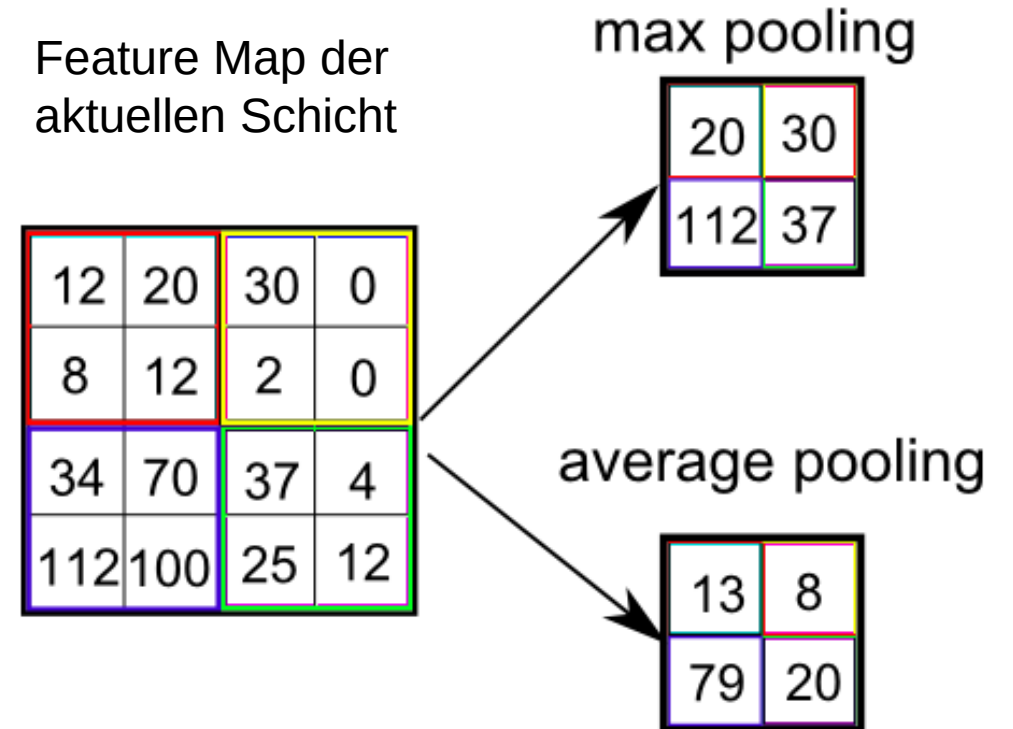


Convolutional Neural Network - CNN

Pooling-Schicht

Pooling-Schicht

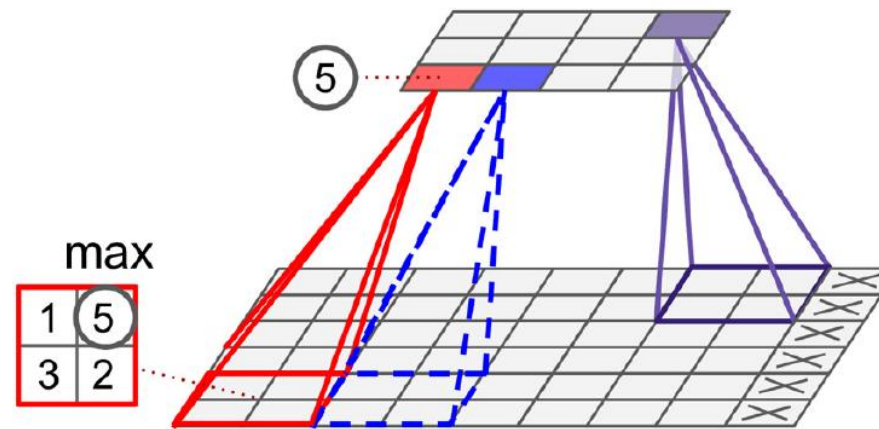
- Aggregiert die Ausgabe einer Convolutional Schicht (nach der Aktivierungsfunktion)
 - Max oder Mean
- Schrumpft die Feature Map
- Einstellungen
 - Pooling-Kernel (z.B. 2x2)
 - Stride (hier: 2)
 - Paddingtyp (hier: keines)
- Anzahl der Feature Maps bleibt erhalten



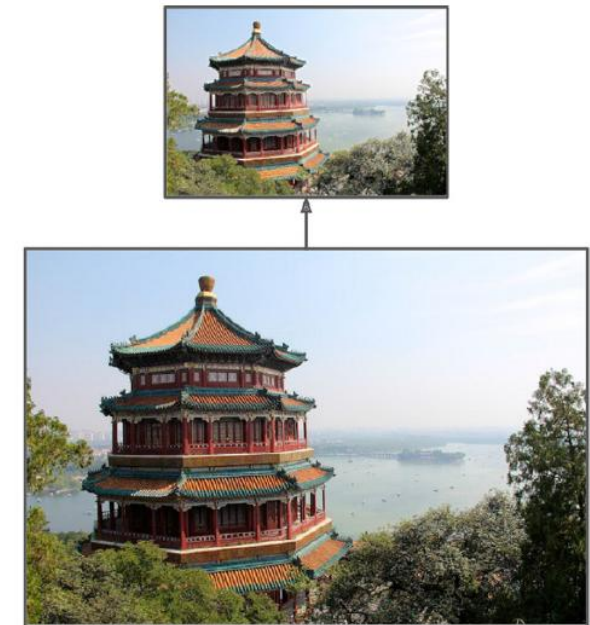
Convolutional Neural Network - CNN

Pooling-Schicht

- Reduziert Berechnungs- und Speicherbedarf
- Subsampling – Neuronen der nachfolgenden Schicht sehen mehr vom Bild bei gleicher Größe des rezeptiven Felds



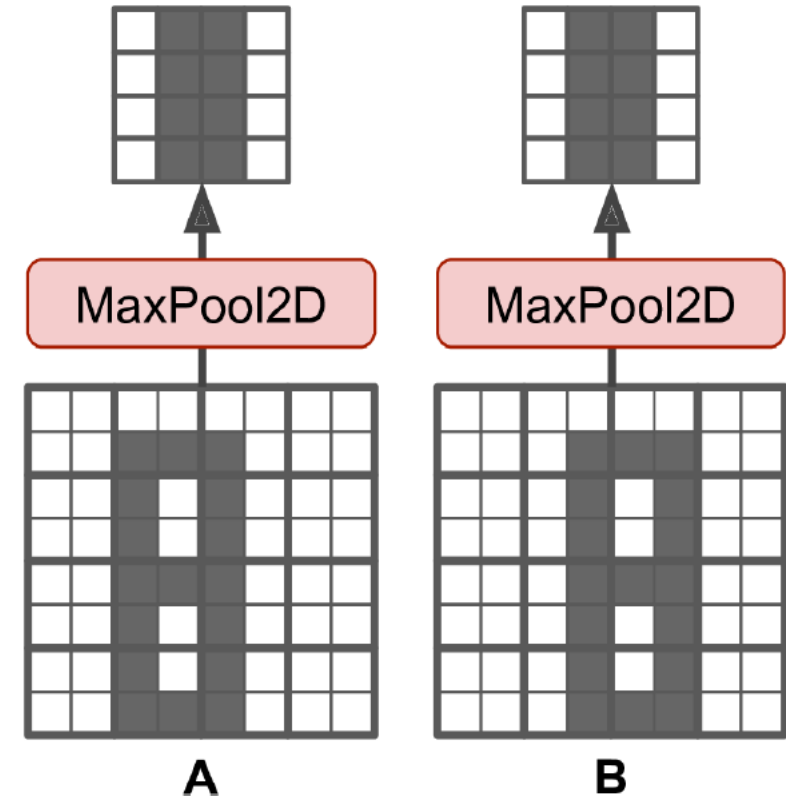
Max Pooling, filter: 2x2, stride = 2, kein Padding



Convolutional Neural Network - CNN

Pooling-Schicht

- Gewährleistet kleine Translations- und Rotationsinvarianz
- Im Beispiel rechts müssen Farben invertiert werden.
 - Weißes Pixel soll Wert 0 haben
 - Schwarzes Pixel soll Wert 255 haben



Max Pooling, filter: 2x2, stride = 2, kein Padding

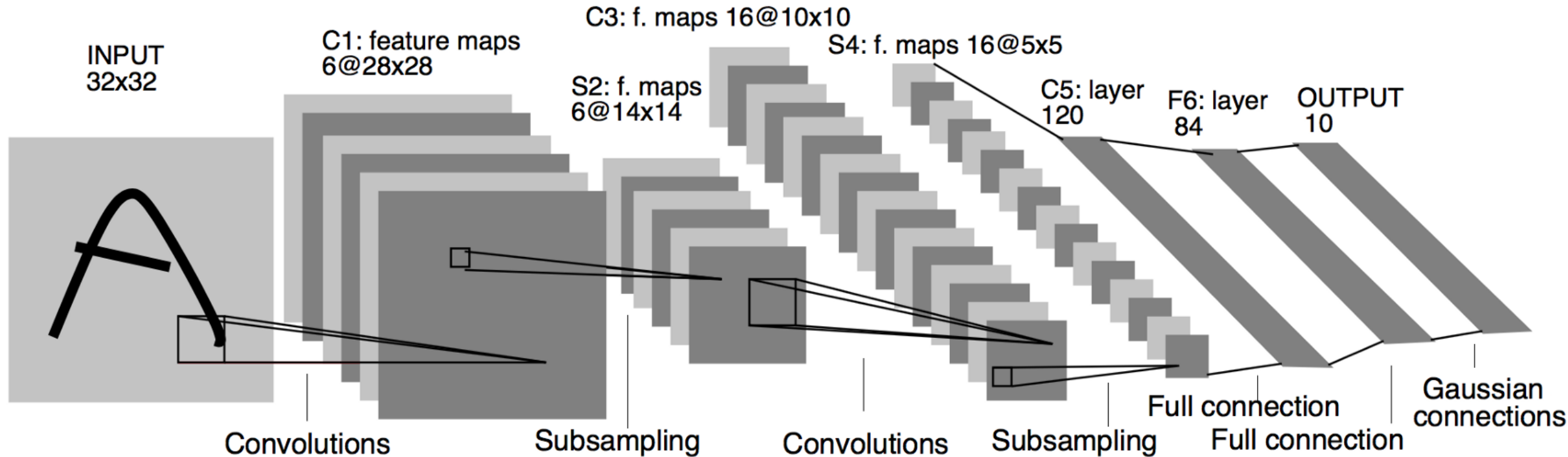
Convolutional Neural Network - CNN

Leitfragen:

- Wie programmiert man CNNs in keras?

Convolutional Neural Network - CNN

LeNet-5 von Yann LeCun (1989)



- Drei CNN-Schichten: C1 (6 Filter), C3 (16 Filter) & C5 (120 Filter) (5x5 Filtergröße, Stride=1, kein padding)
- Zwei Pooling-Schichten: S2 & S4 (Poolingkernel: 2x2, Stride=2)
- Eine Fully-Connected-Schicht: F6 (84 Neuronen)
- Ausgabeschicht mit 10 Neuronen (Erkennung von handgeschriebenen Ziffern: 0 bis 9)

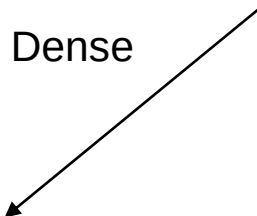
Convolutional Neural Network - CNN

Keras Implementierung des LeNet-5 CNNs

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, AveragePooling2D, Flatten, Dense

model = Sequential([
    Conv2D(filters=6, kernel_size=(5,5), activation='tanh', input_shape=(32,32,1), padding='valid'),
    AveragePooling2D(pool_size=(2,2), strides=2),
    Conv2D(filters=16, kernel_size=(5,5), activation='tanh', padding='valid'),
    AveragePooling2D(pool_size=(2,2), strides=2),
    Conv2D(filters=120, kernel_size=(5,5), activation='tanh', padding='valid'),
    Flatten(),
    Dense(units=84, activation='tanh'),
    Dense(units=10, activation='softmax')])
```

Eingabeschicht wird nicht als eigener Layer modelliert, sondern über `input_shape` definiert



Convolutional Neural Network - CNN

Keras Implementierung des LeNet-5 CNNs

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Conv2D, AveragePooling2D, Flatten, Dense
```

```
model = Sequential([  
    Conv2D(filters=6, kernel_size=(5,5), activation='tanh', input_shape=(32,32,1), padding='valid'),  
    AveragePooling2D(pool_size=(2,2), strides=2),  
    Conv2D(filters=16, kernel_size=(5,5), activation='tanh', padding='valid'),  
    AveragePooling2D(pool_size=(2,2), strides=2),  
    Conv2D(filters=120, kernel_size=(5,5), activation='tanh', padding='valid'),  
    Flatten(),  
    Dense(units=84, activation='tanh'),  
    Dense(units=10, activation='softmax')])
```

Eingabeschicht wird nicht als eigener Layer modelliert, sondern über `input_shape` definiert

Konkateniert die 120 Feature Maps zu einem Vektor; hier ebenfalls 120 dimensional, da Feature Maps des 3. Conv-Layers nur noch 1-dimensional, s. vorherige Folie

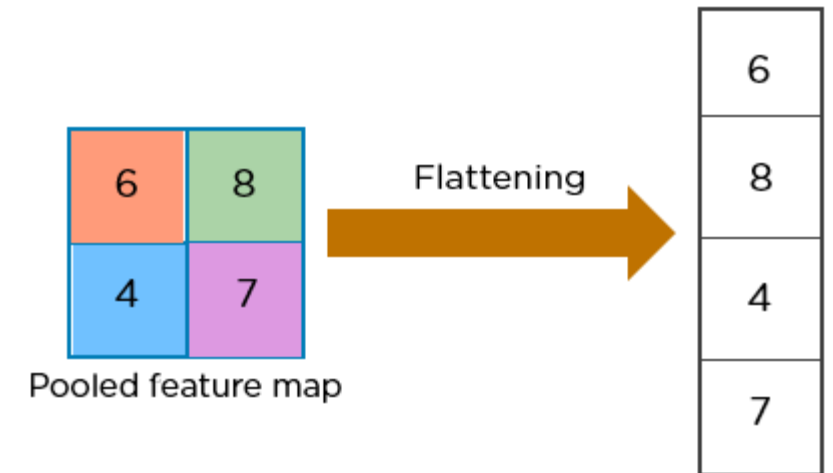
Convolutional Neural Network - CNN

Flatten Layer

```
from tensorflow.keras.layers import Conv2D, AveragePooling2D, Flatten, Dense
```

```
model = Sequential([  
    ...  
    Conv2D(filters=120, kernel_size=(5,5), activation='tanh', padding='valid'),  
    Flatten(),  
    Dense(units=84, activation='tanh'),  
    Dense(units=10, activation='softmax'))])
```

Konkateniert die 120 Feature Maps zu einem Vektor; hier ebenfalls 120 dimensional, da Feature Maps des 3. Conv-Layers nur noch 1-dimensional, s. vorherige Folie



Convolutional Neural Network - CNN

Padding in keras

padding: string, either "valid" or "same" (case-insensitive).

- "valid" means no padding.
- "same" results in padding evenly to the left/right or up/down of the input.

Convolutional Neural Network - CNN

Keras Implementierung eines CNNs

Implementierung einer CNN-Architektur für das Fashion-MNIST Klassifizierungsproblem

```
from functools import partial

DefaultConv2D = partial(keras.layers.Conv2D,
                        kernel_size=3, activation='relu', padding="SAME")

model = keras.models.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    keras.layers.MaxPooling2D(pool_size=2),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    keras.layers.MaxPooling2D(pool_size=2),
    keras.layers.Flatten(),
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=10, activation='softmax'),
])
```

Convolutional Neural Network - CNN

Keras Implementierung eines CNNs

Zusammenfassung des Modells

Wie berechnen sich die Anzahl Parameter?

Layer 1 (kernel_size = 7, s. vorherige Folie):

-
-
-

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 64)	3200
max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 14, 14, 128)	73856
conv2d_2 (Conv2D)	(None, 14, 14, 128)	147584
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_3 (Conv2D)	(None, 7, 7, 256)	295168
conv2d_4 (Conv2D)	(None, 7, 7, 256)	590080
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 256)	0
flatten_2 (Flatten)	(None, 2304)	0
dense_6 (Dense)	(None, 128)	295040
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8256
dropout_4 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 10)	650

Total params: 1,413,834
 Trainable params: 1,413,834
 Non-trainable params: 0

Convolutional Neural Network - CNN

Keras Implementierung eines CNNs

Zusammenfassung des Modells

Wie berechnen sich die Anzahl Parameter?

Layer 1 (kernel_size = 7, s. vorherige Folie):

- $(7*7*1 + 1)*64 = 3200$
- „*1“ da Grauwertbild mit 1 Kanal

■

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 64)	3200
max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 14, 14, 128)	73856
conv2d_2 (Conv2D)	(None, 14, 14, 128)	147584
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_3 (Conv2D)	(None, 7, 7, 256)	295168
conv2d_4 (Conv2D)	(None, 7, 7, 256)	590080
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 256)	0
flatten_2 (Flatten)	(None, 2304)	0
dense_6 (Dense)	(None, 128)	295040
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8256
dropout_4 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 10)	650

Total params: 1,413,834
 Trainable params: 1,413,834
 Non-trainable params: 0

Convolutional Neural Network - CNN

Keras Implementierung eines CNNs

Zusammenfassung des Modells

Wie berechnen sich die Anzahl Parameter?

Layer 1 (kernel_size = 7, s. vorherige Folie):

- $(7*7*1 + 1)*64 = 3200$
- „*1“ da Grauwertbild mit 1 Kanal

Layer 2:

■

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 64)	3200
max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 14, 14, 128)	73856
conv2d_2 (Conv2D)	(None, 14, 14, 128)	147584
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_3 (Conv2D)	(None, 7, 7, 256)	295168
conv2d_4 (Conv2D)	(None, 7, 7, 256)	590080
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 256)	0
flatten_2 (Flatten)	(None, 2304)	0
dense_6 (Dense)	(None, 128)	295040
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8256
dropout_4 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 10)	650

Total params: 1,413,834
 Trainable params: 1,413,834
 Non-trainable params: 0

Convolutional Neural Network - CNN

Keras Implementierung eines CNNs

Zusammenfassung des Modells

Wie berechnen sich die Anzahl Parameter?

Layer 1 (kernel_size = 7, s. vorherige Folie):

- $(7*7*1 + 1)*64 = 3200$
- „*1“ da Grauwertbild mit 1 Kanal

Layer 2:

- $(3*3*64+1)*128 = 73856$

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 64)	3200
max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 14, 14, 128)	73856
conv2d_2 (Conv2D)	(None, 14, 14, 128)	147584
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_3 (Conv2D)	(None, 7, 7, 256)	295168
conv2d_4 (Conv2D)	(None, 7, 7, 256)	590080
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 256)	0
flatten_2 (Flatten)	(None, 2304)	0
dense_6 (Dense)	(None, 128)	295040
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8256
dropout_4 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 10)	650

Total params: 1,413,834
 Trainable params: 1,413,834
 Non-trainable params: 0

Künstliche Neuronale Netze - KNN

Keras Implementierung eines CNNs

1. Kompilieren und trainieren des Modells

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam", metrics=["accuracy"])  
history = model.fit(X_train, y_train, epochs=10, validation_data=(X_valid, y_valid))
```

2. Auswerten des Modells auf den Testdaten

```
model.evaluate(X_test, y_test)
```

```
10000/10000 - loss: 0.3467 - accuracy: 0.8816
```

Künstliche Neuronale Netze - KNN

Keras Implementierung eines CNNs

1. Kompilieren und trainieren des Modells

```
model.compile(loss="sparse_categorical_crossentropy", optimizer="nadam", metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=10, validation_data=(X_valid, y_valid))
```

2. Auswerten des Modells auf den Testdaten

```
model.evaluate(X_test, y_test)
```

10000/10000 - loss: 0.3467 - accuracy: 0.8816

3. Vorhersage auf „neue“ Daten

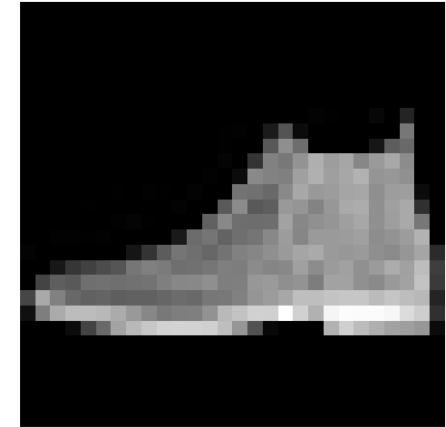
```
y_proba = model.predict(X_test[:3])
y_proba.round(2)
```

```
array([[0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.01, 0.   , 0.99],
       [0.   , 0.   , 1.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   ],
       [0.   , 1.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   ]],
      dtype=float32)
```

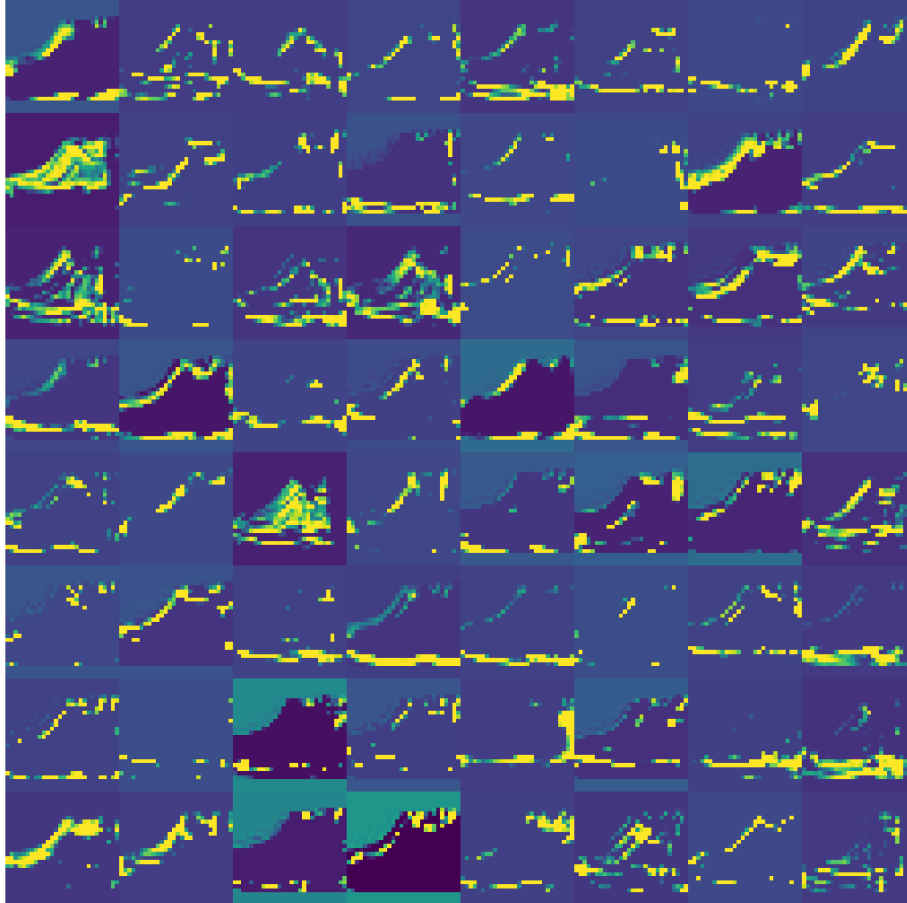
Künstliche Neuronale Netze - KNN

Visualisierung der Feature Maps eines CNN - Übung

Eingabebild: Stiefelette

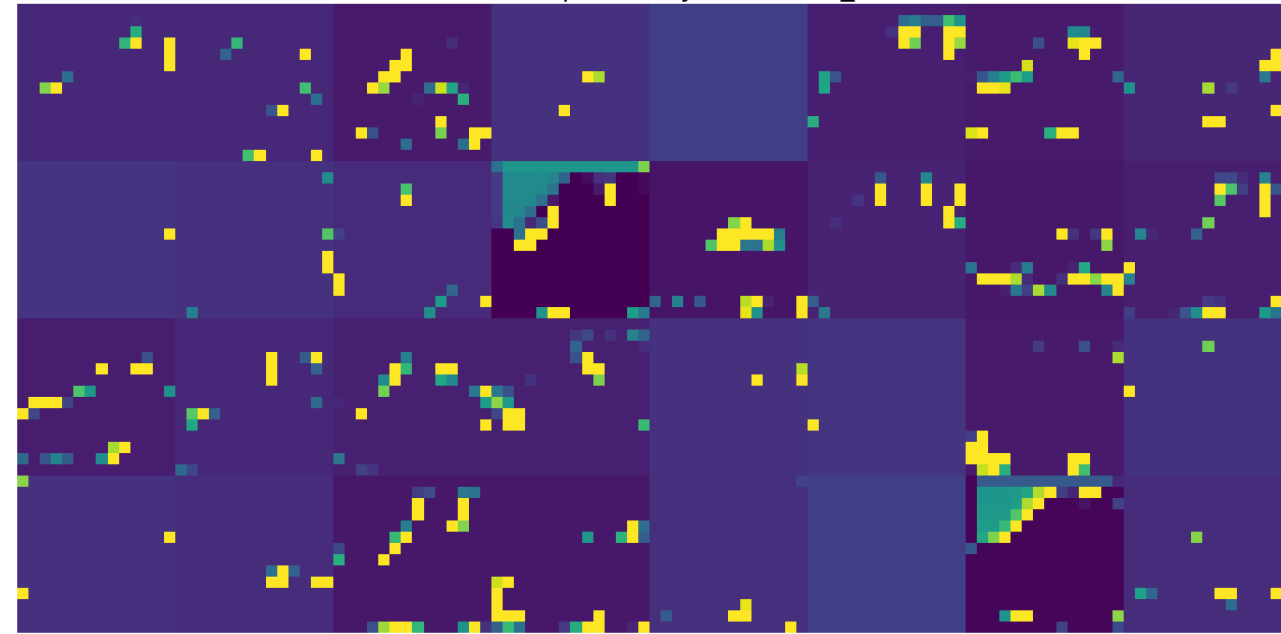


Feature Maps für Layer: conv2d_10



[05_fashion_mnist_feature_maps.ipynb](#)

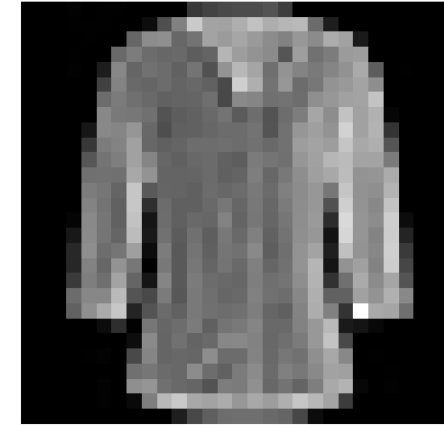
Feature Maps für Layer: conv2d_11



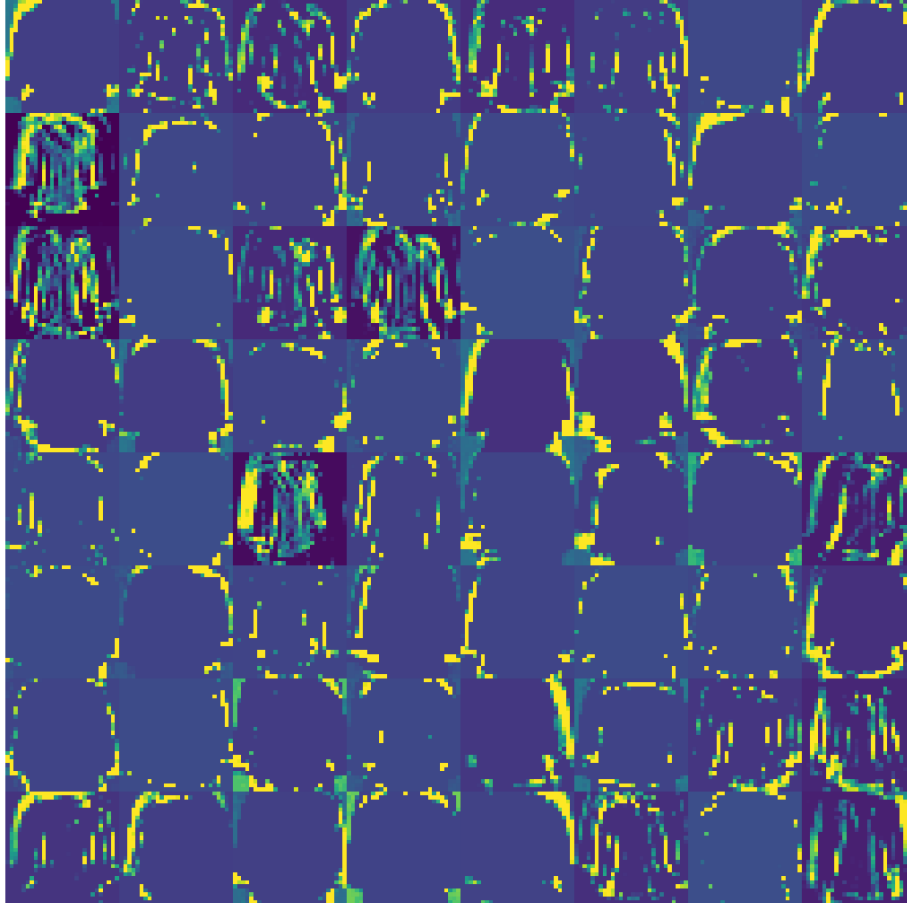
Künstliche Neuronale Netze - KNN

Visualisierung der Feature Maps eines CNN - Übung

Eingabebild: Shirt

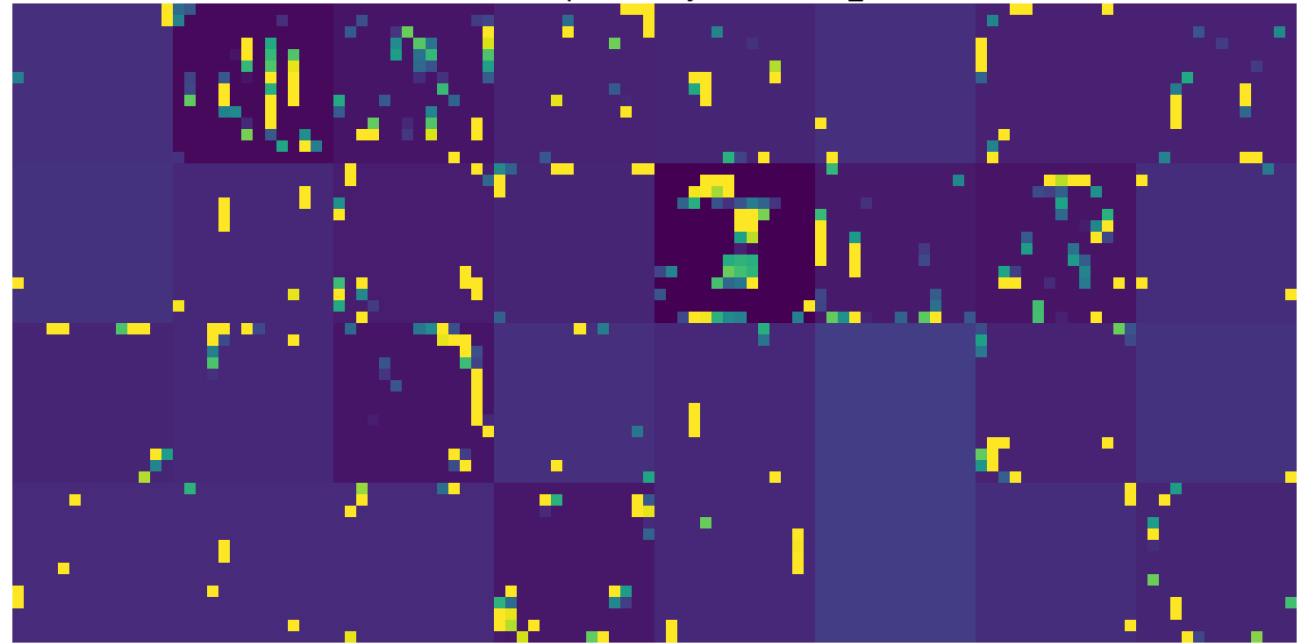


Feature Maps für Layer: conv2d_10



[05_fashion_mnist_feature_maps.ipynb](#)

Feature Maps für Layer: conv2d_11



Künstliche Neuronale Netze - KNN

Keras Implementierung eines CNNs - Übung

Ausgehend vom Notebook [02_fashion_mnist_cnn.ipynb](#) probieren Sie

- verschiedene Architekturen,
- Hyperparameter,
- Regularisierungstechniken aus

um bessere Ergebnisse auf den Validierungsdaten zu erzielen.

Falls Sie weights & biases zur Protokollierung Ihrer Ergebnisse nutzen möchten, können Sie auch mit dem Notebook [03_fminst_cnn_wb.ipynb](#) starten.

Zusammenfassung

- CNNs unterscheiden sich zu fully-connected neuronalen Netzen durch
 - Neuron einer Schicht ist nur mit Neuronen der vorherigen Schicht verbunden, die im rezeptiven Feld des Neurons sind
 - Neuronen zur Berechnung einer Feature Map teilen sich die Gewichte; diese Gewichte werden in einem Filter gespeichert

■

■

■

■

■

Zusammenfassung

- CNNs unterscheiden sich zu fully-connected neuronalen Netzen durch
 - Neuron einer Schicht ist nur mit Neuronen der vorherigen Schicht verbunden, die im rezeptiven Feld des Neurons sind
 - Neuronen zur Berechnung einer Feature Map teilen sich die Gewichte; diese Gewichte werden in einem Filter gespeichert
- Jede Convolutional Schicht kann mehrere Filter haben, die je eine Feature Map berechnen
- Anwendung eines Filters auf die Eingabedaten kann man sich vorstellen als:
 - Jedes Neuron berechnet die gewichtete Summe der Pixel in seinem rezeptiven Feld
 - Filter wird mit Eingabebild gefaltet
-

Zusammenfassung

- CNNs unterscheiden sich zu fully-connected neuronalen Netzen durch
 - Neuron einer Schicht ist nur mit Neuronen der vorherigen Schicht verbunden, die im rezeptiven Feld des Neurons sind
 - Neuronen zur Berechnung einer Feature Map teilen sich die Gewichte; diese Gewichte werden in einem Filter gespeichert
- Jede Convolutional Schicht kann mehrere Filter haben, die je eine Feature Map berechnen
- Anwendung eines Filters auf die Eingabedaten kann man sich vorstellen als:
 - Jedes Neuron berechnet die gewichtete Summe der Pixel in seinem rezeptiven Feld
 - Filter wird mit Eingabebild gefaltet
- Pooling Schicht verkleinert die Feature Map einer Convolutional Schicht durch Subsampling

Nächste Vorlesung und Fragen

- Rekurrente Netze, Transformer Netze und Natural Language Processing
- Fragen?